



Széchenyi István Katolikus Technikum és Gimnázium

Szoftverfejlesztő és -tesztelő projektfeladat

Szikszi FM

Készítették:

Szabó Dániel,

Csépányi Gergely

Ózd, 2026

Tartalom

Bevezetés.....	3
Felhasználói dokumentáció	4
Rendszerkövetelmény	4
Hardver:.....	4
Szoftver:	4
Webalkalmazás indítása	4
Webalkalmazás használata	5
Általános információk a weboldalról.....	5
Főoldal	5
Bejelentkezés/regisztráció oldal	6
Műsorok oldal.....	7
Műsorvezetők oldal.....	8
Rólunk oldal	8
Fejlesztői dokumentáció.....	10
Alkalmazott fejlesztői és csoportmunka eszközök.....	10
Adatbázis.....	10
Frontend	10
Backend.....	10
Adatbázis	10
Modell leírása (adatmezők - típusok - leírásuk).....	10
Táblák, kapcsolatok	11
E-K diagram	14
Frontend.....	15
Alkalmazás frontend részének részletes leírása	15
Backend	33
Alkalmazás backend részének részletes leírása.....	33
Továbbfejlesztési lehetőségek	58
Irodalomjegyzék.....	59

Bevezetés

A projekt célja egy **komplex weboldal** létrehozása az **iskolarádió** számára, amely modern, könnyen kezelhető és informatív felületet biztosít mind a diákok, mind a tanárok számára. Az oldal célja, hogy digitális platformot kínáljon az iskolai rádióműsorok, hírek, események és archív tartalmak bemutatására, ezáltal növelve a közösségi élet aktivitását és átláthatóságát.

A weboldal nem csupán információs felületként szolgál, hanem interaktív funkciókat is kínál majd — például műsorrend megtekintését, élő adások hallgatását, valamint a diákok által készített tartalmak közzétételét. A projekt során a tervezés, fejlesztés és tesztelés lépései is megvalósulnak, különös figyelmet fordítva a **felhasználói élményre** a **reszponzív dizájnra**, valamint a **biztonságos és megbízható működésre**.

<https://github.com/szaboaprofi/Szikszi-FM.git>

Felhasználói dokumentáció

Rendszerkövetelmény

Hardver:

- Asztali gép/laptop: Legalább 4GB RAM, 2 magos CPU, internetkapcsolat
- Mobil eszköz: Android 8.0+ vagy iOS 12.0+ támogatása

Szoftver:

- VS Code v1.96.0 vagy újabb
- Node.js v22.13.0 vagy újabb
- XAMPP 8.1.12 vagy újabb
- Composer 2.8.4 vagy újabb

Webalkalmazás indítása

A webalkalmazás elindításához el kell indítanunk a XAMPP alkalmazást és elindítani az Apache, illetve MySQL részét. Ezt követően meg kell nyitni egy Visual Studio Code alkalmazást és megnyitni a BACKEND mappán belül lévő source mappát, majd CTRL+Shift+ö billentyű kombináció lenyomásával előhozzuk a terminált ennek a PowerShell részében kell kiadnunk a parancsot, ami létrehozza, illetve feltölti az adatbázis: `php artisan migrate --seed`. Így az adatbázisunkat létrehoztuk már csak el kell indítani a szerveret, amit a következő paranccsal tudunk végrehajtani: `php artisan serve`

Amint ezzel kész vagyunk nyitni kell még egy Visual Studio Code alkalmazást és megnyitni a FRONTEND mappát. Szintén nyitni kell egy terminált a CTRL+Shift+ö billentyű kombinációval, majd ki kell adni az `npm install` parancsot a terminal cmd részében, amikor a terminal befejezi a folyamatot ki kell adnunk a következő parancsot: `ng s -o`. Ez a parancs elindítja a webalkalmazást, amit kidob egy böngésző ablakban.

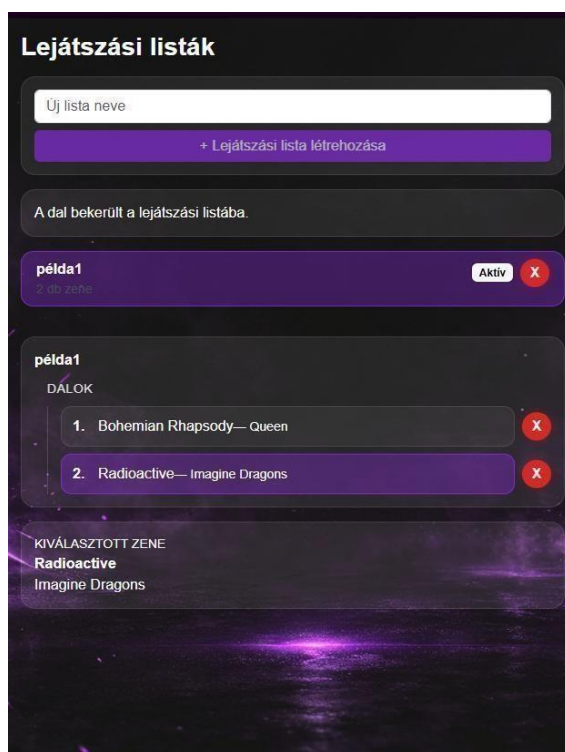
Webalkalmazás használata

Általános információk a weboldalról

Az oldalunk egy iskolarádió gördülékenyebb működését segíti elő, melyben a bejelentkezett felhasználók láthatják a heti lejátszott zenéket, illetve aktuális műsorvezetőket. Ezen felül saját lejátszási listát hozhatnak létre.

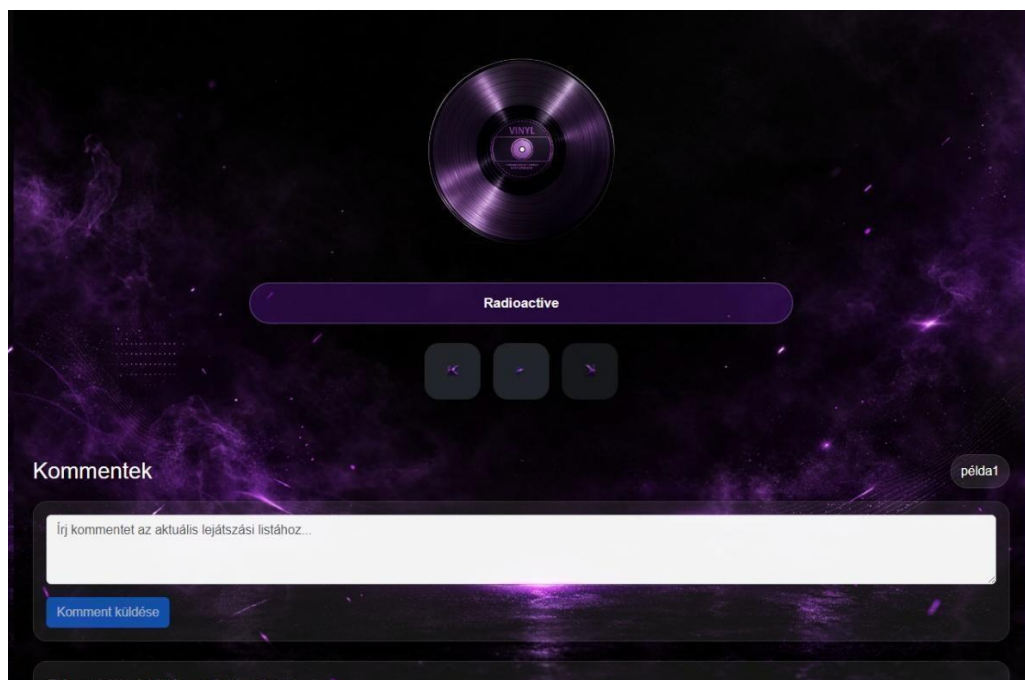
Főoldal

A főoldalt 3 részre lehet osztani, van a baloldali rész, ahol a lejátszási listákat tudunk létrehozni, illetve látjuk a már létrehozott lejátszási listáinkat



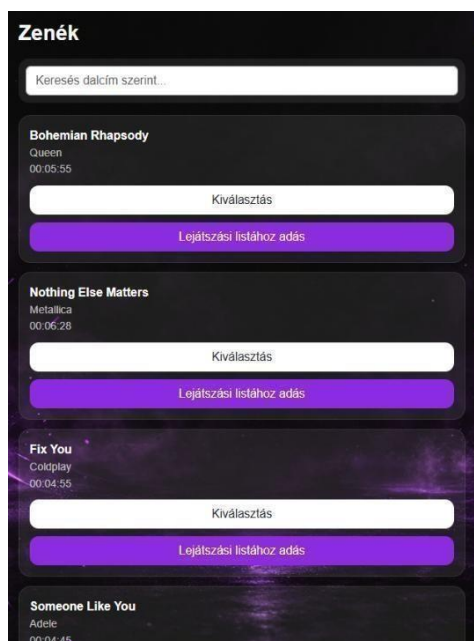
1. kép: Főoldal, lejátszási listák oszlop

Középen található a zene lejátszó, illetve alatta a lejátszási listák komment felülete itt tudunk lépkedni a lejátszási lista zenéi között a jobb, illetve bal nyíllal.



2. Kép: Főoldal, középső rész

Jobboldalt pedig a zenéket látjuk, amelyeket hozzáadhatunk lejátszási listához, a felső keresősávban pedig zene cím alapján kereshetünk zenékre.



3. Kép: Főoldal, Jobboldali zenék oszlop

Bejelentkezés/regisztráció oldal

A bejelentkezés oldalon jelentkezhetünk be fiókunkba amennyiben rendelkezünk fiókkal, ha nem egy gombnyomással válthatunk a regisztráció oldalra, ahol automatikusan bejelentkeztet minket az oldala regisztrációt követően.

4. Kép:

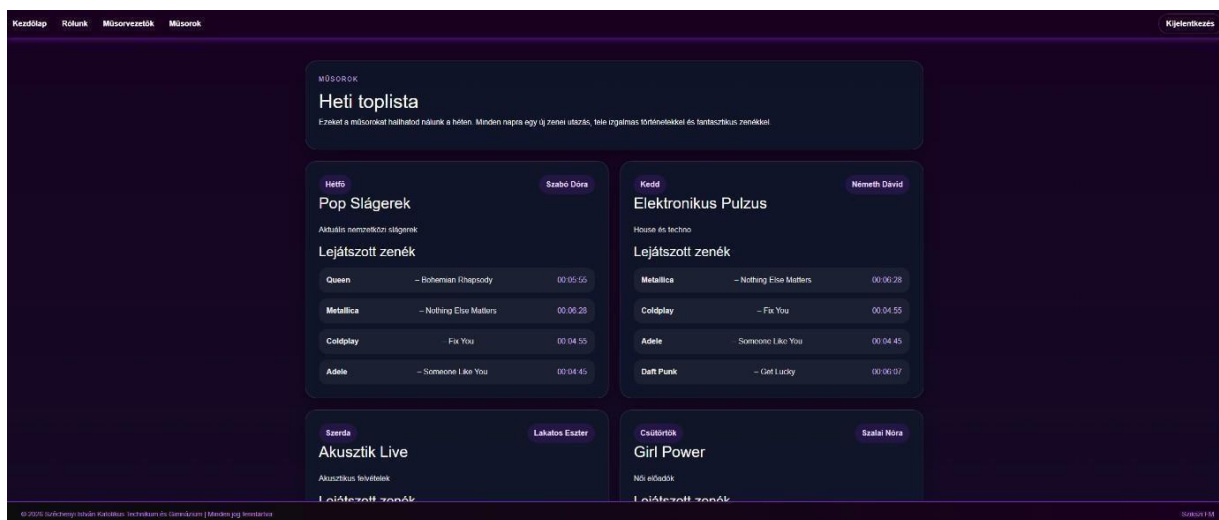
Bejelentkezés fül

5. Kép:

Regisztráció fül

Műsorok oldal

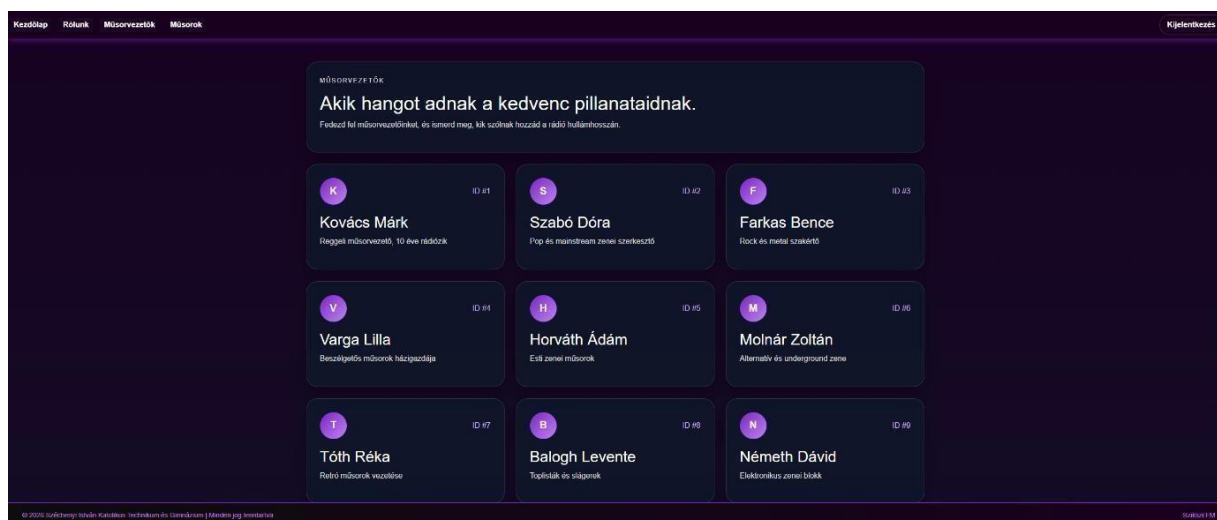
A műsorok oldalon láthatjuk a heti műsorlistát, hogy ki volt/lesz az adott nap műsorvezetője, melyik zenék voltak/lesznek aznap.



6. Kép: Műsorok oldal

Műsorvezetők oldal

A műsorvezetők oldalon láthatjuk a műsorvezetőket, és az általuk vezetett műsorokat.

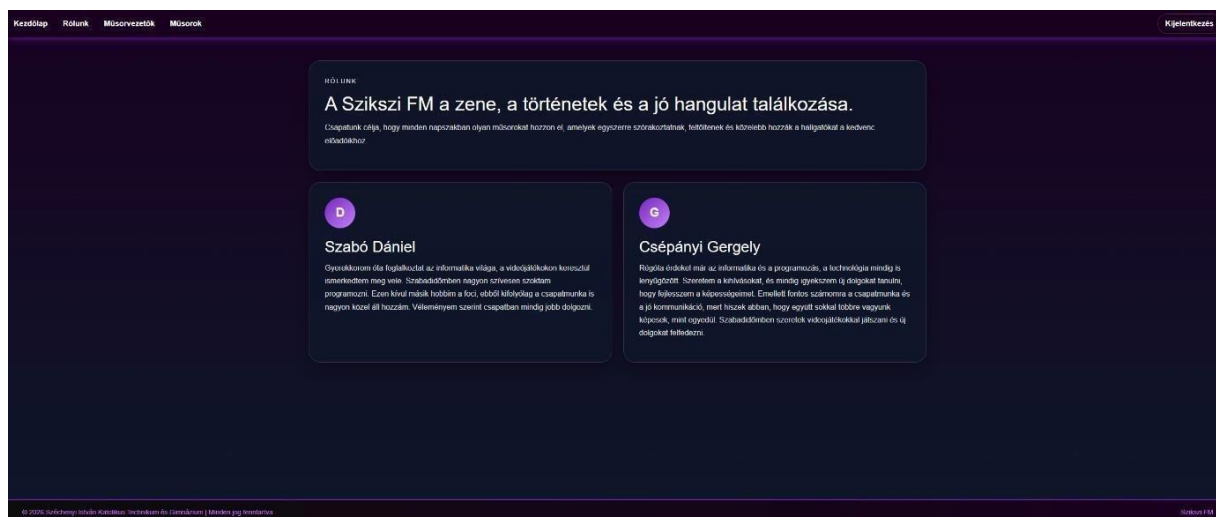


Kép: műsorvezetők oldal

7.

Rólunk oldal

arólunk oldalon lehet rólunk, a fejlesztőkről egy rövid bemutatkozást olvasni.



8. Kép: Rólunk oldal

Fejlesztői dokumentáció

Alkalmazott fejlesztői és csoportmunka eszközök

Git/GitHub: Project tárolása és verziómenedzselés

Discord: Kommunikáció

Trello: Feladatok elosztása a csapattagok között

Adatbázis

Visual Studio Code: SQL kód megírása

XAMPP: SQL kód tesztelése, Apache webservert futtatása

MySQL: adatbázis tárolása

Frontend

Visual Studio Code: A weboldal kódjának megírása

Angular: Keretrendszer a weboldalhoz

Figma: A weboldal megtervezése

Backend

Visual Studio Code: Laravel api, kontrollerek megírása és tesztelése

XAMPP: Adatbázis lokális futtatása és kezelése

Laravel: Keretrendszer az apihoz

Adatbázis

Modell leírása (adatmezők - típusok - leírásuk)

A feladat egy **adatbázis létrehozása egy webalkalmazáshoz** volt. A munka folyamata három fő lépésből állt:

1. **Adatbázis megtervezése** – először felmértük, milyen adatokra van szükség a webalkalmazás működéséhez, és meghatároztuk a táblák szerkezetét, a mezőket és a kapcsolódásokat.
2. **Adatbázis elkészítése** – a tervek alapján létrehoztuk a táblákat, beállítottuk az elsődleges és idegen kulcsokat, valamint feltöltöttük az adatokat tesztcélokra.
3. **EK-diagram készítése** – a kapcsolatok és struktúrák átláthatósága érdekében elkészítettük az **entitás–kapcsolat diagramot**, amely vizuálisan szemlélteti az adatbázis szerkezetét és a táblák közötti kapcsolatokat. Ez a folyamat biztosítja, hogy az adatbázis jól szervezett, konzisztens és könnyen kezelhető legyen a webalkalmazás számára.

Táblák, kapcsolatok

Felhasználók

Mező	Típus	Leírás
id	int	egyedi azonosító és elsődleges kulcs
felhasznalonev	varchar, 255 karakter hosszú	felhasználó neve
email	varchar, 255 karakter hosszú	felhasználó email-je
jelszo	varchar, 255 karakter hosszú	felhasználó jelszava
szerep	varchar, 255 karakter hosszú	felhasználó szerepköre
letrehozva	date	regisztráció ideje

A felhasználók tábla tárolja az összes rendszerben regisztrált személy adatait. Ez az alap tábla, amelyhez több másik tábla idegen kulccsal kapcsolódik.

Műsorvezetők

Mező	Típus	Leírás
------	-------	--------

id	int	egyedi azonosító és elsődleges kulcs
felhasznalo_id	int	idegen kulcs, hivatkozik a felhasznalok táblára
nev	varchar, 255 karakter hosszú	műsorvezető neve
bemutakozas	varchar, 255 karakter hosszú	műsorvezető bemutatkozása

A műsorvezetők a felhasználók egy speciális csoportját alkotják. Minden műsorvezetőhöz tartozik egy felhasználói fiók.

Műsorok

Mező	Típus	Leírás
id	int	egyedi azonosító és elsődleges kulcs
cim	varchar, 255 karakter hosszú	műsor címe
leiras	varchar, 255 karakter hosszú	műsor leírása
musorvezeto_id	int	idegen kulcs, hivatkozik a musorvezetok táblára
letrehozva	date	műsor létrehozásának ideje

A műsorok tábla tartalmazza az egyes rádióműsorokat, amelyeket egy adott műsorvezető vezet.

Dalok

Mező	Típus	Leírás
id	int	egyedi azonosító és elsődleges kulcs
eloado	varchar, 255 karakter hosszú	előadó neve
cim	varchar, 255 karakter hosszú	dal címe
hossza	time	dal hossza

A dalok tábla a lejátszható zeneszámokat tartalmazza, amelyek több műsorban is felhasználhatók. Lejátszási lista

Mező	Típus	Leírás
id	int	egyedi azonosító és elsődleges kulcs
nev	varchar, 255 karakter hosszú	lejátszási lista neve
felhasznalo_id	int	idegen kulcs, hivatkozik a felhasznalok táblára
playlist_id	int	idegen kulcs, hivatkozik a lejatszolistak táblára
dal_id	int	idegen kulcs, hivatkozik a dalok táblára
sorrend_szam	int	lejátszási lista sorszáma
musor_id	int	idegen kulcs, hivatkozik a musorok táblára
letrehozva	datetime	lejátszási lista létrehozásának ideje

A lejátszási lista kapcsolótábla, amely meghatározza, hogy egy műsorban milyen dalok és milyen sorrendben hangzanak el.

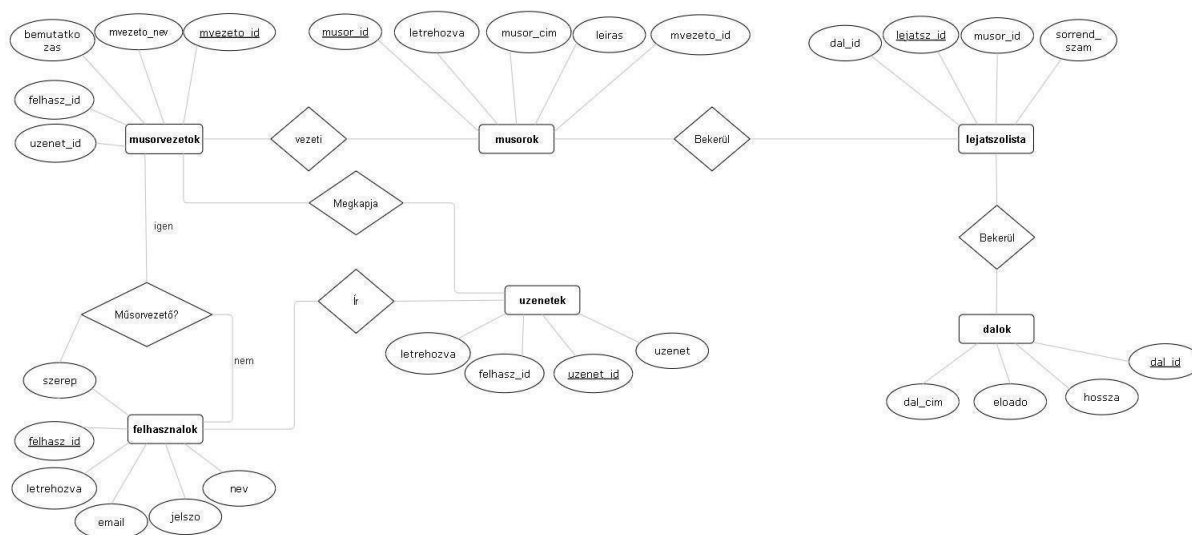
Üzenetek

Mező	Típus	Leírás
id	int	egyedi azonosító és elsődleges kulcs
felhasznalo_id	int	idegen kulcs, hivatkozik a felhasznalok táblára
lejatszalista_id	int	idegen kulcs, hivatkozik a lejatszolistak táblára

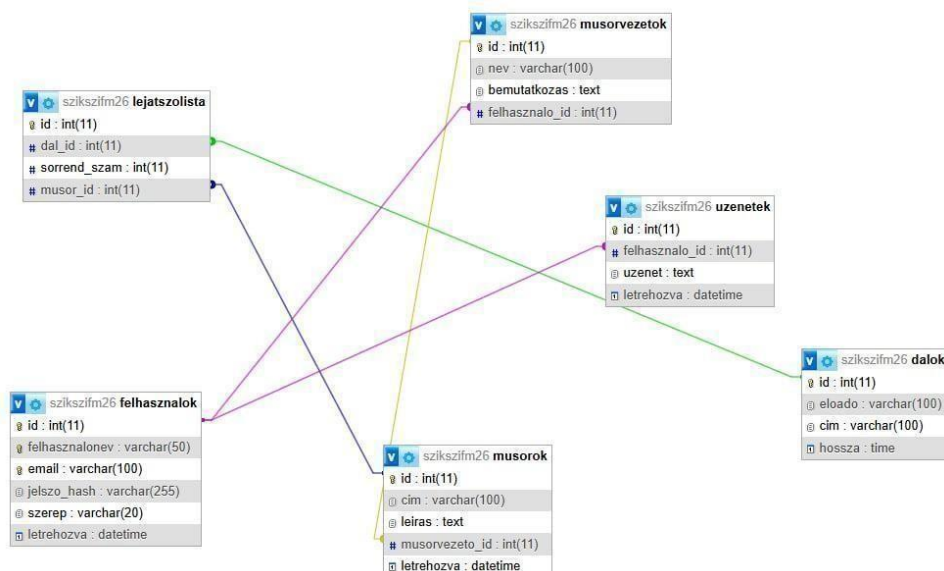
uzenet	text	felhasználó üzenete
letrehozva	datetime	üzenet létrehozásának ideje

Az üzenetek tábla a hallgatók visszajelzéseit és hozzászólásait tárolja.

E-K diagram



9. Kép: EK Diagramm



Frontend

Alkalmazás frontend részének részletes leírása

A dokumentáció ezen része a webalkalmazás frontend részének működését és felépítését mutatja be. A frontend az alkalmazás azon része, amely közvetlen kapcsolatban áll a felhasználóval, tehát ez biztosítja a grafikus felületet, az interakciókat és az adatok megjelenítését. A frontend fő feladata, hogy a backendből érkező adatokat feldolgozza és megjelenítse, valamint a felhasználói műveleteket (például kattintások, bevitel, navigáció) kezelje. Emellett biztosítja az alkalmazás állapotának kezelését és a különböző nézetek közötti váltást. Az alkalmazás Angular keretrendszerrel készült, amely lehetővé teszi a komponens alapú felépítést. Ez azt jelenti, hogy a felhasználói felület kisebb, egymástól elkülönített részekből épül fel, amelyek külön-külön is kezelhetők és újra felhasználhatók. A projekt modern Angular megoldásokat alkalmaz, például standalone komponenseket és signal alapú állapotkezelést.

Reszponzív megjelenés (mobil és tablet nézet)

A webalkalmazás reszponzív kialakítással készült, amely biztosítja, hogy az oldal különböző képernyőméreteken (mobil, tablet, asztali gép) is megfelelően jelenjen meg. A reszponzivitás megvalósítása során rugalmas rácsszerkezetet (grid layout), valamint százalékos és relatív méretezést alkalmaztunk. A komponensek elrendezése automatikusan igazodik a képernyő méretéhez. Mobil nézetben a több oszlopos elrendezés egyoszloposra vált, így a tartalom egymás alatt jelenik meg. A navigáció és az egyes elemek mérete is igazodik a kisebb kijelzőhöz, hogy könnyen kezelhető maradjon érintőképernyőn is. Tablet nézetben az elrendezés részben megtartja az oszlopos struktúrát, azonban az elemek mérete és elhelyezése optimalizálva van a közepes kijelzőméretekhez. A reszponzív működés különösen fontos a felhasználói élmény szempontjából, mivel az alkalmazás mobil eszközökön is kényelmesen használható.



11. kép: Főoldal - Mobil nézet

Bejelentkezés

Menü kiválasztása

Bejelentkezés

Lépj be és kezeld a saját játékosai listáidat.

Email

Jelszó

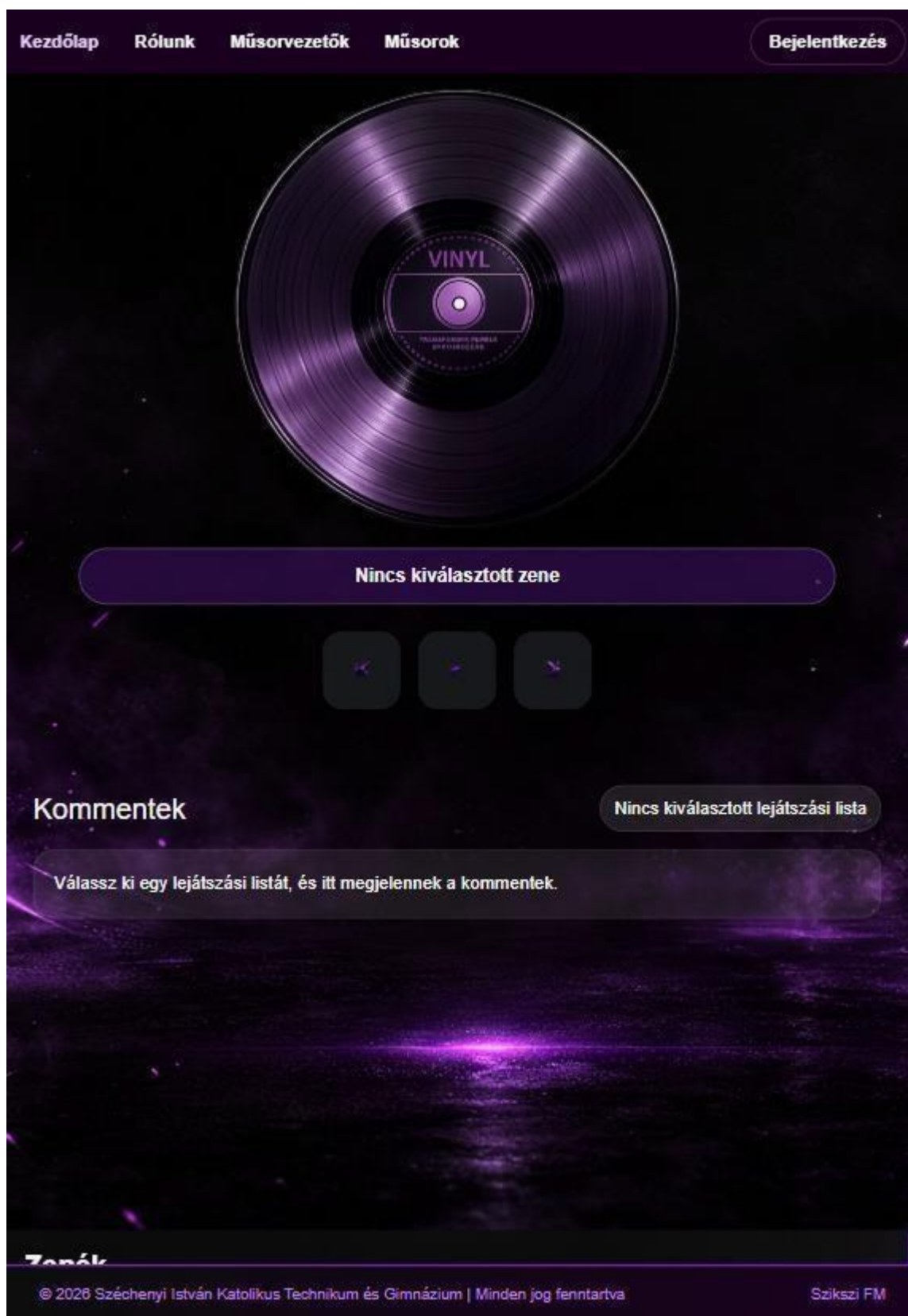
Bejelentkezés

Nincs még fiókod? **Regisztráció**

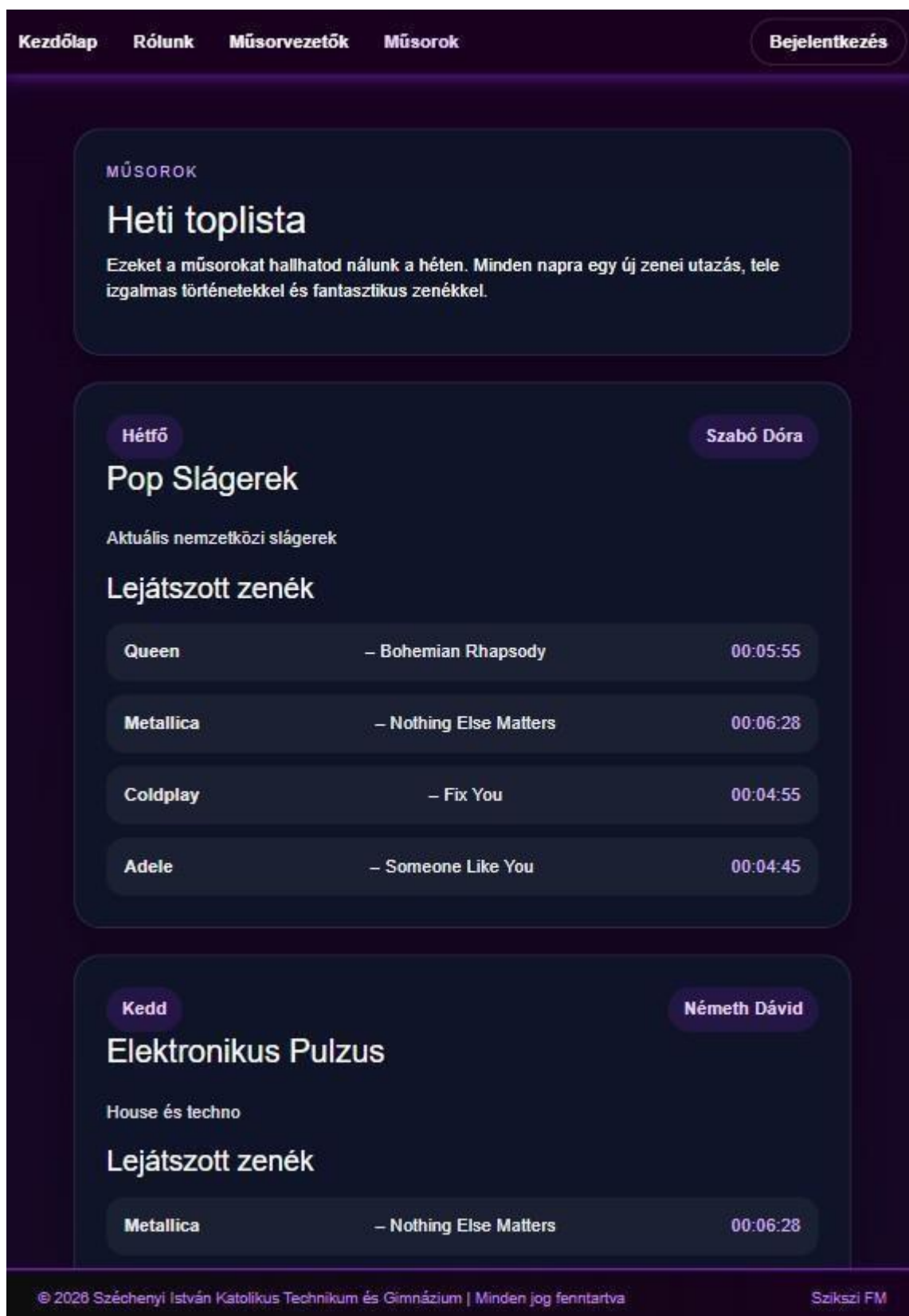
© 2026 Széchenyi István Katolikus
Technikum és Gimnázium | Minden jog
fenntartva

Szükszi
FM

12. Kép: Bejelentkezés - Mobil nézet

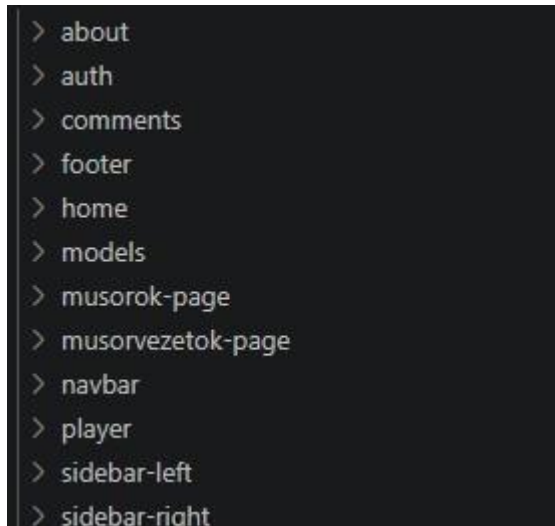


13. Kép: Főoldal - Tablet nézet



14. Kép: Műsorok oldal - Tablet nézet

Komponens alapú felépítés



15. Kép: Komponensek

A frontend több különálló komponensből épül fel, amelyek mindegyike egy adott funkcióért felel. Ez a megközelítés lehetővé teszi a kód átláthatóbbá tételét és a fejlesztés egyszerűsítését. Minden komponens tartalmaz egy HTML sablont, amely a megjelenítésért felel, valamint egy TypeScript fájlt, amely a működés logikáját tartalmazza. A komponensek közötti kommunikáció jellemzően service-eken keresztül történik, amelyek az adatkezelést végzik.

Fő alkalmazás

```
<div class="app-shell">
  <app-navbar></app-navbar>

  <main class="app-main">
    <router-outlet></router-outlet>
  </main>

  <app-footer></app-footer>
</div>
```

16. Kép: App komponens

Az alkalmazás központi eleme az App komponens, amely az egész felület vázát biztosítja. Ez a komponens tartalmazza a navigációs sávot, az aktuális oldalt megjelenítő részt és a lábléceket. A navigációs sáv minden oldalon látható, és lehetővé teszi a felhasználó számára az oldalak közötti váltást. A RouterOutlet nevű elem felel azért, hogy az aktuális URL alapján a megfelelő komponens jelenjen meg. A lábléc az oldal alján helyezkedik el, és általában statikus információkat tartalmaz.

Home oldal működése

```

<div class="container-fluid main-container">
  <div class="row h-100">
    <div class="col-12 col-lg-3 p-0 center-scroll position-relative">
      <app-sidebar-left></app-sidebar-left>
    </div>

    <div class="col-12 col-lg-6 p-0 center-scroll position-relative">
      <app-player></app-player>
      <app-comments></app-comments>
    </div>

    <div class="col-12 col-lg-3 p-0 center-scroll position-relative">
      <app-sidebar-right></app-sidebar-right>
    </div>
  </div>
</div>

```

16. Kép: Home oldal

A Home oldal az alkalmazás központi része, ahol a legtöbb funkció elérhető. Ez az oldal több kisebb komponensből áll össze, amelyek együtt biztosítják a teljes funkcionalitást. A Home oldal figyeli a bejelentkezett felhasználó állapotát, és ennek megfelelően tölti be a szükséges adatokat, például a felhasználóhoz tartozó playlisteket. Ha a felhasználó változik, az oldal automatikusan frissíti a megjelenített adatokat.

Az oldal felépítése több részre osztható. A bal oldalon a playlist kezelés található, a jobb oldalon a zenék listája jelenik meg, középen a lejátszó, alul pedig a kommentek. Playlist kezelés (SidebarLeft)

```

1 <div class="sidebar-left p-3 h-100">
2   <h3 class="fw-bold mb-3">Lejátszási listák</h3>
3   @if (!user()) {
4     <p class="hint">Jelentkezz be, hogy saját listákat hozhass létre.</p>
5   } @else {
6     <div class="create-card mb-3">
7       <input
8         type="text"
9         class="form-control mb-2"
10        [ngModel]="newPlaylistName()"
11        (ngModelChange)="onPlaylistNameChange($event)"
12        placeholder="Új lista neve"
13      />
14      @if (playlistNameError()) {
15        <div class="playlist-error mb-2">{{ playlistNameError() }}</div>
16      }
17      <button class="btn create-btn w-100" [disabled]="!canCreate()" (click)="createPlaylist()">
18        + Lejátszási lista létrehozása
19      </button>
20    </div>
21  }
22  @if (statusMessage()) {
23    <div class="status-box mb-3">{{ statusMessage() }}</div>
24  }
25  <div class="list-group playlist-list">
26    @for (lista of playlists(); track lista.id) {
27      <button
28        type="button"
29        class="list-group-item list-group-item-action sidebar-link"
30        [class.active-playlist]="selectedPlaylistId() === lista.id"
31        (click)="selectPlaylist(lista.id)">
32        <div class="d-flex justify-content-between align-items-start gap-2">
33          <div>
34            <strong>{{ lista.nev }}</strong>
35            <div class="small text-light-emphasis">{{ lista.tetelek.length }} db zene</div>
36          </div>
37          <div class="playlist-actions">
38            @if (selectedPlaylistId() === lista.id) {
39              <span class="badge text-bg-light">Aktív</span>
40            }
41            @if (user()) {
42              <button
43                type="button"
44                class="delete-x-btn"
45                title="Lejátszási lista törlése"
46                aria-label="Lejátszási lista törlése"
47                (click)="$event.stopPropagation(); deletePlaylist(lista.id)">
48                X

```

17. Kép: SidebarLeft

A bal oldali panel a felhasználó saját lejátszási listáinak kezelésére szolgál. Ez a komponens lehetővé teszi a lejátszási listák megjelenítését, létrehozását és törlését. A felhasználó új listát hozhat létre egy név megadásával, kiválaszthatja a meglévő lejátszási listái közül azt, amelyikkel dolgozni szeretne, illetve törölheti a már nem szükséges listákat. Emellett lehetőség van arra is, hogy egy playlistből eltávolítsunk egy adott dalt. Ez a komponens szorosan együttműködik a központi állapotkezeléssel, így minden változás azonnal megjelenik a felületen.

Zenék kezelése (SidebarRight)

```

<div class="sidebar-right p-3 h-100 position-relative right-scroll">
  <h3 class="fw-bold mb-3">Zenék</h3>

  <div class="search-box mb-3">
    <input
      type="text"
      class="form-control"
      placeholder="Keresés dalcím szerint..."
      [ngModel]="searchTerm()"
      (ngModelChange)="searchTerm.set($event)"
    />
  </div>

  <div class="music-grid">
    @for (music of filteredMusics(); track music.id) {
      <div class="music-card" [class.selected]="playlistState.selectedSong()?.id === music.id">
        <div>
          <div class="music-title">{{ music.cim }}</div>
          <div class="music-meta">{{ music.eloado }}</div>
          <div class="music-meta">{{ music.hossza }}</div>
        </div>

        <button type="button" class="btn select-btn" (click)="selectSong(music)">
          Kiválasztás
        </button>
        <button type="button" class="btn add-btn" (click)="addSelectedSong(music)">
          Lejátszási listához adás
        </button>
      </div>
    } @empty {
      <div class="empty-search">Nincs találat erre a dalcímre.</div>
    }
  </div>
</div>

```

18. Kép: SidebarRight

A jobb oldali panel a zenék böngészésére szolgál. Itt jelenik meg az összes elérhető dal, amelyeket a felhasználó megtekinthet és kiválaszthat. A komponens lehetőséget biztosít keresésre is, így a felhasználó könnyen megtalálhatja a kívánt zenét. A keresés a frontend oldalon történik, tehát a már betöltött adatok szűrésével valósul meg. A kiválasztott dal hozzáadható az aktuálisan kiválasztott playlisthez, így a felhasználó könnyedén összeállíthatja saját zenei listáit. Lejátszó (Player)

```

<div class="player-shell text-center my-4">
  <div class="record-wrapper">
    
  </div>

  <div class="current-song-label">{{ currentSongTitle() }}</div>

  <div class="player-controls">
    <button class="btn btn-dark control-btn" [disabled]="!canGoPrevious()" (click)="previousSong()">
      
    </button>

    <button class="btn btn-dark control-btn" [disabled]="!selectedSong()" (click)="togglePlay()">
      <img [src]="isPlaying() ? 'kepek/stop.png' : 'kepek/Play.png'" alt="Lejátszás" />
    </button>

    <button class="btn btn-dark control-btn" [disabled]="!canGoNext()" (click)="nextSong()">
      
    </button>
  </div>
</div>

```

19. Kép: Player

A Player komponens a kiválasztott dal megjelenítéséért és kezeléséért felel. Megjeleníti az aktuálisan kiválasztott dal címét, és lehetőséget biztosít a lejátszás vezérlésére. A felhasználó elindíthatja vagy megállíthatja a lejátszást, valamint léptethet az előző vagy következő dalra. A lejátszó jelenleg nem valódi audiolejátszást végez, hanem a kiválasztott dal állapotát kezeli, és a felhasználói felületen jeleníti meg az információkat.

Kommentek kezelése (Comments)

```

<div class="comments-shell mt-5">
  <div class="d-flex justify-content-between align-items-center mb-3 flex-wrap gap-2">
    <h3 class="m-0">Komentek</h3>
    <div class="selected-playlist-pill">
      {{ selectedPlaylist()?.nev ?? 'Nincs kiválasztott lejátszási lista' }}
    </div>
  </div>

  @if (selectedPlaylist()) {
    <div class="comment-form card mb-4">
      <div class="card-body">
        <textarea
          class="form-control mb-3"
          rows="3"
          placeholder="Írj kommentet az aktuális lejátszási listához..."
          [ngModel]="newComment()"
          (ngModelChange)="newComment.set($event)"
        ></textarea>
        <button class="btn btn-primary" [disabled]="!canSend()" (click)="addComment()">
          Komment küldése
        </button>
      </div>
    </div>

    @for (comment of comments(); track comment.id) {
      <div class="card my-3 comment-card">
        <div class="card-body">
          <div class="comment-header">
            <strong>{{ comment.felhasznalo?.felhasznalonev ?? ('Felhasználó #' + comment.felhasznalo_id) }}</strong>
            <span>{{ comment.letrehozva }}</span>
          </div>
          <p class="mb-0">{{ comment.uzenet }}</p>
        </div>
      </div>
    } @empty {
      <div class="empty-comments">Ehhez a lejátszási listához még nincs komment.</div>
    }
  } @else {
    <div class="empty-comments">Válassz ki egy lejátszási listát, és itt megjelennek a kommentek.</div>
  }
</div>

```

20. Kép: Comments komponens

A Comments komponens a playlistekhez tartozó kommentek megjelenítésére és kezelésére szolgál. Megjeleníti az aktuálisan kiválasztott playlisthez tartozó hozzászólásokat, és lehetőséget biztosít új komment írására. Komment hozzáadására csak akkor van lehetőség, ha a felhasználó be van jelentkezve, és ki van választva egy playlist. Ez biztosítja, hogy a kommentek mindig egy adott playlisthez kapcsolódjanak.

Bejelentkezés és regisztráció (Auth)

```

@if (isLoginMode()) {
  <form [formGroup]="loginForm" (ngSubmit)="submitLogin()">
    <label>Email</label>
    <input type="email" formControlName="email" placeholder="Email cím" />
    @if (loginForm.controls.email.touched && loginForm.controls.email.hasError('required')) {
      <div class="field-error">Az email cím megadása kötelező.</div>
    }
    @if (loginForm.controls.email.touched && loginForm.controls.email.hasError('email')) {
      <div class="field-error">Adj meg egy valós email címet.</div>
    }

    <label>Jelszó</label>
    <input type="password" formControlName="password" placeholder="Jelszó" />
    @if (loginForm.controls.password.touched && loginForm.controls.password.hasError('required')) {
      <div class="field-error">A jelszó megadása kötelező.</div>
    }
    @if (loginForm.controls.password.touched && loginForm.controls.password.hasError('minlength')) {
      <div class="field-error">A jelszónak legalább 6 karakterből kell állnia.</div>
    }

    <button type="submit" [disabled]="loading()">{{ loading() ? 'Beléptetés...' : 'Bejelentkezés' }}</button>
  </form>

```

21. Kép: Bejelentkezési űrlap és validáció

Az Auth komponens a felhasználók hitelesítéséért felel, vagyis itt történik a bejelentkezés és a regisztráció kezelése. A komponens egy űrlapot használ, amely lehetővé teszi a felhasználói adatok megadását és ellenőrzését. A képen látható, hogy a bejelentkezési űrlap két fő mezőt tartalmaz: email és jelszó. Ezek a mezők a felhasználó által megadott adatokat fogadják, amelyeket a rendszer feldolgoz a bejelentkezés során. Az űrlap elküldését egy esemény kezeli, amely meghívja a megfelelő függvényt, és elindítja a bejelentkezési folyamatot. A rendszer ellenőrzi a megadott adatokat, és szükség esetén hibaüzenetet jelenít meg, ha valamelyik mező hibásan vagy hiányosan van kitöltve.

```

@else {
<form [formGroup]="registerForm" (ngSubmit)="submitRegister()">
  <label>Felhasználónév</label>
  <input type="text" formControlName="felhasznalonev" placeholder="Felhasználónév" />
  @if (registerForm.controls.felhasznalonev.touched && registerForm.controls.felhasznalonev.hasError('required')) {
    <div class="field-error">A felhasználónév megadása kötelező.</div>
  }
  @if (registerForm.controls.felhasznalonev.touched && registerForm.controls.felhasznalonev.hasError('minlength')) {
    <div class="field-error">A felhasználónév legalább 3 karakter legyen.</div>
  }

  <label>Email</label>
  <input type="email" formControlName="email" placeholder="Email cím" />
  @if (registerForm.controls.email.touched && registerForm.controls.email.hasError('required')) {
    <div class="field-error">Az email cím megadása kötelező.</div>
  }
  @if (registerForm.controls.email.touched && registerForm.controls.email.hasError('email')) {
    <div class="field-error">Adj meg egy érvényes email címet.</div>
  }

  <label>Jelszó</label>
  <input type="password" formControlName="password" placeholder="Jelszó" />
  @if (registerForm.controls.password.touched && registerForm.controls.password.hasError('required')) {
    <div class="field-error">A jelszó megadása kötelező.</div>
  }
  @if (registerForm.controls.password.touched && registerForm.controls.password.hasError('minlength')) {
    <div class="field-error">A jelszónak legalább 6 karakter hosszúnak kell lennie.</div>
  }

  <label>Jelszó újra</label>
  <input type="password" formControlName="confirmPassword" placeholder="Jelszó újra" />
  @if (registerForm.controls.confirmPassword.touched && registerForm.controls.confirmPassword.hasError('required')) {
    <div class="field-error">Írd be újra a jelszót.</div>
  }
  @if (registerForm.controls.confirmPassword.touched && registerForm.controls.confirmPassword.hasError('passwordMismatch')) {
    <div class="field-error">A két jelszó nem egyezik meg.</div>
  }

  <button type="submit" [disabled]="loading()">{{ loading() ? 'Mentés...' : 'Regisztráció' }}</button>
</form>

```

22. Kép: Validáció működése

A validáció fontos része az űrlap működésének. Az Angular ellenőrzi, hogy az email mező ki van-e töltve, illetve, hogy megfelelő formátumú-e. A jelszó esetében szintén kötelező a kitöltés, és minimum hossz is meg van adva (6 karakter). Ha a felhasználó hibás adatot ad meg, akkor a rendszer azonnal hibaüzenetet jelenít meg a mező alatt. A hibakezelés feltételes megjelenítéssel történik, például csak akkor jelenik meg a hibaüzenet, ha a mezőt már módosította a felhasználó, és az nem felel meg a feltételeknek. Ez javítja a felhasználói élményt, mert nem jelennek meg feleslegesen hibák már az oldal betöltésekor. A bejelentkezés gomb dinamikusan változik az állapot függvényében. Ha folyamatban van a belépés, akkor a gomb szövege „Beléptetés...” lesz, és a gomb letiltásra kerül. Ez megakadályozza, hogy a felhasználó többször egymás után elküldje az űrlapot.

Műsorok oldal

```

<section class="shows-page">
  <div class="container py-5">
    <div class="section-head mb-4">
      <p class="eyebrow">Műsorok</p>
      <h1>Heti toplista</h1>
      <p>
        Ezeket a műsorokat hallhatod nálunk a héten. Minden napra egy új zenei utazás, tele izgalmas történetekkel és fantasztikus zenékkal.
      </p>
    </div>

    @if (hetiMűsorok$ | async; as hetiMűsorok) {
      <div class="row g-4">
        @for (napiMűsor of hetiMűsorok; track napiMűsor.nap) {
          <div class="col-12 col-xl-6">
            <article class="show-card h-100">
              <div class="card-header-row">
                <div>
                  <span class="day-pill">{{ napiMűsor.nap }}</span>
                  <h2>{{ napiMűsor.cím }}</h2>
                </div>
                <span class="host-name">{{ napiMűsor.műsorvezeto }}</span>
              </div>

              <p class="description">{{ napiMűsor.leiras }}</p>

              <h3>Lejátszott zenék</h3>
              <ul class="song-list">
                @for (dal of napiMűsor.dalok; track dal.id) {
                  <li>
                    <strong>{{ dal.eloadó }}</strong> - {{ dal.cím }}
                    <span>{{ dal.hossza }}</span>
                  </li>
                }
              </ul>
            </article>
          </div>
        }
      </div>
    }
  </div>
</section>

```

23. Kép: Műsorok oldal

A műsorok oldal egy összetettebb logikát tartalmazó rész, amely több különböző adatforrást kombinál. Az oldal egy heti műsortervet állít össze a rendelkezésre álló adatok alapján. Ez a komponens nem csupán megjeleníti az adatokat, hanem feldolgozza és rendszerezi is azokat, így dinamikusan generált tartalmat jelenít meg.

Műsorvezetők oldal

```

<section class="listing-page">
  <div class="container py-5">
    <div class="section-head mb-4">
      <p class="eyebrow">Műsorvezetők</p>
      <h1>Akik hangot adnak a kedvenc pillanataidnak.</h1>
      <p>Fedezd fel műsorvezetőinket, és ismerd meg, kik szólnak hozzád a rádió hullámhosszán.</p>
    </div>

    @if (musorvezetok$ | async; as musorvezetok) {
      <div class="row g-4">
        @for (musorvezeto of musorvezetok; track musorvezeto.id) {
          <div class="col-12 col-md-6 col-xl-4">
            <article class="info-card h-100">
              <div class="card-top">
                <div class="avatar">{{ musorvezeto.nev.charAt(0) }}</div>
                <span class="tag">ID #{{ musorvezeto.id }}</span>
              </div>
              <h2>{{ musorvezeto.nev }}</h2>
              <p>{{ musorvezeto.bemutakozas }}</p>
            </article>
          </div>
        }
      </div>
    }
  </div>
</section>

```

24. Kép: Műsorvezetők oldal

A műsorvezetők oldal a rendszerben szereplő műsorvezetők megjelenítésére szolgál. Az oldal célja, hogy a felhasználó áttekinthető formában lássa, kik tartoznak az alkalmazáshoz, és rövid leírást kapjon róluk. A felület felső részén egy cím és egy rövid bemutató szöveg található, amely leírja az oldal tartalmát. Ez alatt jelenik meg a műsorvezetők listája. Az adatok betöltése aszinkron módon történik, és az Angular @if valamint async pipe segítségével kerülnek megjelenítésre. Ez biztosítja, hogy az oldal csak akkor jelenítse meg a tartalmat, amikor az adatok már rendelkezésre állnak. A lista megjelenítése egy @for ciklussal történik, amely végigmegy a műsorvezetők tömbjén, és minden elemhez egy külön kártyát hoz létre. Ezek a kártyák tartalmazzák a műsorvezető nevét, azonosítóját és bemutatkozását. A megjelenítés reszponzív rácsszerkezetben történik, így az oldal különböző képernyőméretekken is jól használható marad.

Állapotkezelés - AuthService

```

import { Injectable, computed, signal } from '@angular/core';
import { User } from '../models/user.model';

@Injectable({ providedIn: 'root' })
export class AuthStateService {
  private readonly storageKey = 'szikszi_user';
  readonly user = signal<User | null>(this.readStoredUser());
  readonly isLoggedIn = computed(() => this.user() !== null);
  readonly displayName = computed(() => this.user()?.felhasznalonev ?? 'Bejelentkezés');

  setUser(user: User): void {
    this.user.set(user);
    localStorage.setItem(this.storageKey, JSON.stringify(user));
  }

  logout(): void {
    this.user.set(null);
    localStorage.removeItem(this.storageKey);
  }

  private readStoredUser(): User | null {
    const raw = localStorage.getItem(this.storageKey);
    if (!raw) {
      return null;
    }

    try {
      return JSON.parse(raw) as User;
    } catch {
      localStorage.removeItem(this.storageKey);
      return null;
    }
  }
}

```

25. Kép: AuthStateService

Ez a service a felhasználói állapot kezeléséért felel. Tárolja az aktuális felhasználót, és meghatározza, hogy a felhasználó be van-e jelentkezve. A service gondoskodik arról is, hogy a felhasználó adatai megmaradjanak az oldal újratöltése után is, például a localStorage használatával. Emellett kezeli a kijelentkezést is.

PlaylistStateService

```

createPlaylist(userId: number, nev: string): Observable<{ success: boolean; message: string }> {
  return this.http.post<{ success: boolean; message: string }>(`${this.apiUrl}/create-playlist`, {
    felhasznalo_id: userId,
    nev,
  }).pipe(
    tap((response) => this.statusMessage.set(response.message)),
    tap(() => {
      this.loadPlaylists(userId).subscribe({ next: () => undefined, error: () => undefined });
    })
  );
}

addSongToPlaylist(playlistId: number, dalId: number, userId: number): Observable<{ success: boolean; message: string }> {
  return this.http.post<{ success: boolean; message: string }>(`${this.apiUrl}/${playlistId}/songs`, {
    dal_id: dalId,
  }).pipe(
    tap((response) => this.statusMessage.set(response.message)),
    tap(() => {
      this.loadPlaylists(userId).subscribe({ next: () => undefined, error: () => undefined });
    })
  );
}

removeSongFromPlaylist(playlistId: number, tetelId: number, userId: number): Observable<{ success: boolean; message: string }> {
  return this.http.delete<{ success: boolean; message: string }>(`${this.apiUrl}/${playlistId}/songs/${tetelId}`).pipe(
    tap((response) => this.statusMessage.set(response.message)),
    tap(() => {
      this.loadPlaylists(userId).subscribe({ next: () => undefined, error: () => undefined });
    })
  );
}

deletePlaylist(playlistId: number, userId: number): Observable<{ success: boolean; message: string }> {
  return this.http.delete<{ success: boolean; message: string }>(`${this.apiUrl}/${playlistId}/delete-playlist`).pipe(
    tap((response) => this.statusMessage.set(response.message)),
    tap(() => {
      this.loadPlaylists(userId).subscribe({ next: () => undefined, error: () => undefined });
    })
  );
}

```

26. Kép: PlaylistStateService

Ez a frontend egyik legfontosabb eleme, amely a playlistekhez kapcsolódó adatokat és állapotokat kezeli. Tárolja a playlisteket, a kiválasztott playlistet, a kiválasztott dalt, valamint a kommenteket. Emellett biztosítja a dalok közötti navigációt és a különböző műveletek végrehajtását. Ez a service biztosítja, hogy a felhasználói felület mindig szinkronban legyen az aktuális állapottal.

Service-ek szerepe

```

TS auth-state.service.ts
TS comment-service.spec.ts
TS comment-service.ts
TS music-service.spec.ts
TS music-service.ts
TS musor-service.ts
TS musorvezeto-service.ts
TS playlist-service.spec.ts
TS playlist-service.ts
TS playlist-state.service.ts

```

27. Kép: Service-ek

A service-ek az adatkezelésért és a backenddel való kommunikációért felelnek. Ezek biztosítják, hogy a frontend hozzáférjen a szükséges adatokhoz. A különböző service-ek különböző típusú adatokat kezelnek, például felhasználók, zenék, playlistek, vagy kommentek. Ez a réteg biztosítja az alkalmazás működésének hátterét, és elkülöníti a logikát a megjelenítéstől. Routing

```
export const routes: Routes = [
  { path: '', component: Home },
  { path: 'rolunk', component: About },
  { path: 'musorvezetok', component: MusorvezetokPage },
  { path: 'musorok', component: MusorokPage },
  { path: 'auth', component: Auth },
  { path: '**', redirectTo: '' },
];
```

28. Kép: Routing

A frontend alkalmazásban az oldalak közötti navigációt az Angular Router kezeli. A routing lényege, hogy az alkalmazás az URL alapján dönti el, melyik komponens jelenjen meg a képernyőn. Ennek köszönhetően a felhasználó különböző oldalak között tud váltani úgy, hogy közben az alkalmazás nem töltődik újra teljesen, csak a megfelelő tartalom cserélődik ki. A csatolt kódrészletben a route-ok egy tömbben vannak definiálva, ahol minden útvonalhoz hozzá van rendelve egy komponens. Ez a megoldás átláthatóvá teszi a navigációs logikát, mert egy helyen jól látható, hogy melyik URL melyik oldalhoz tartozik.

A path: '' beállítás a kezdőoldalt jelenti. Ez azt mondja meg, hogy ha a felhasználó az alkalmazás gyökércímére lép, akkor a Home komponens töltődjön be. Ez gyakorlatilag az alkalmazás alapértelmezett nyitóoldala, ahová a felhasználó először érkezik.

A path: 'rolunk' útvonal az About komponenshez tartozik. Ez az oldal általában a projektről vagy az oldal céljáról ad információt, tehát inkább statikus tartalmat jelenít meg. A routing itt azt biztosítja, hogy a „Rólunk” menüpontra kattintva a megfelelő komponens jelenjen meg.

A path: 'musorvezetok' útvonal a MusorvezetokPage komponenst tölti be. Ez az oldal a műsorvezetők megjelenítésére szolgál, vagyis külön route-ot kapott, hogy jól elkülönüljön a többi tartalomtól. Ugyanez a logika működik a path: 'musorok' esetében is, ahol a MusorokPage komponens jelenik meg, tehát a műsorok külön saját oldalt kaptak.

A path: 'auth' útvonal az Auth komponenshez tartozik. Ez a route a bejelentkezési és regisztrációs funkciókat kezeli. Külön útvonalon szerepel, mert ez az alkalmazás egyik fontos, jól elkülöníthető funkcionális része. Így a felhasználó közvetlenül is elérheti a hitelesítéshez tartozó oldalt.

A kódban szerepel egy path: '**' beállítás is, amely az úgynevezett wildcard route. Ennek az a szerepe, hogy elkapjon minden olyan URL-t, ami a korábban definiált útvonalak egyikéhez sem tartozik. Ilyenkor az alkalmazás nem hibára fut, hanem visszairányítja a felhasználót a kezdőoldalra a redirectTo: '' beállítás segítségével. Ez felhasználói szempontból hasznos, mert elkerülhető vele, hogy egy hibás vagy nem létező cím miatt üres vagy rossz oldal jelenjen meg.

Az alábbi ábra a tesztfuttatás eredményét mutatja a terminálban, ahol látható, hogy minden teszt sikeresen lefutott.

Az alkalmazás végfelhasználói működésének ellenőrzésére Playwright alapú end-to-end (E2E) tesztelés került alkalmazásra. A tesztek futtatása az ng e2e paranccsal történik. A Playwright lehetővé teszi az alkalmazás több böngészőbe történő tesztelését: Chromium, Firefox, WebKit.

E2E teszteredmények:

Lefuttatott tesztek száma: 3

Sikeres tesztek: 3

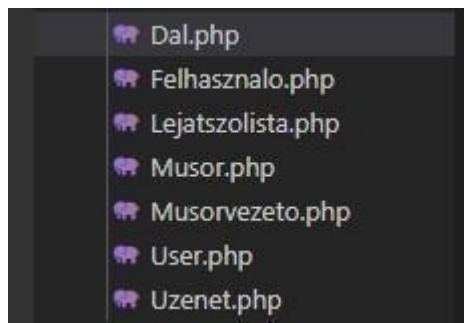
A tesztek minden böngészőben sikeresen lefutottak, ami igazolja az alkalmazás kompatibilitását és stabil működését.

A Vitest és a Playwright alkalmazásával egy modern és hatékony tesztelési környezet került kialakításra. Az egységtesztek és E2E tesztek együttesen biztosítják az alkalmazás megbízható működését, valamint támogatják a további fejlesztést és karbantartást.

Backend

Alkalmazás backend részének részletes leírása

Modellek:



30. Kép: Modellek

Az adatbázisból kifolyólag 7 db Modellre volt szükségünk

A Dal osztály a dalok adatbázistáblához tartozó modell. Ez az osztály teszi lehetővé, hogy a táblában lévő adatokat objektumorientált módon érjük el, anélkül, hogy nyers SQL lekérdezéseket kellene írni.

```
class Dal extends Model
{
    use HasFactory;
    public $table='dalok';
    public $timestamps=false;
    public $guarded=[];
}
```

31. Kép: Modell példa

Egyes részek feladatai:

'class Dal extends Model'

- Ez jelzi, hogy a Dal egy modell, vagyis egy adatbázistábla logikai reprezentációja.

'use HasFactory;'

- Lehetővé teszi, hogy factorykat használjunk a modellhez, például teszteléshez vagy mintaadatok generálásához.

'public \$table = 'dalok';'

- Alapértelmezés szerint a Laravel a modell nevét többes számban használná (dals), ezért itt explicit módon megadjuk, hogy a modell a dalok nevű táblához tartozik.

```
'public $timestamps = false;'
```

- Kikapcsolja az automatikus created_at és updated_at mezők kezelését. Erre akkor van szükség, ha az adatbázistábla nem tartalmaz ilyen oszlopokat.

```
'public $guarded = [];'
```

- Ez azt jelenti, hogy nincs védett mező, vagyis minden oszlop tömegesen feltölthető (mass assignment). Ez egyszerűsíti az adatmentést, de csak akkor biztonságos, ha a bemeneti adatokat már megfelelően validáltuk.

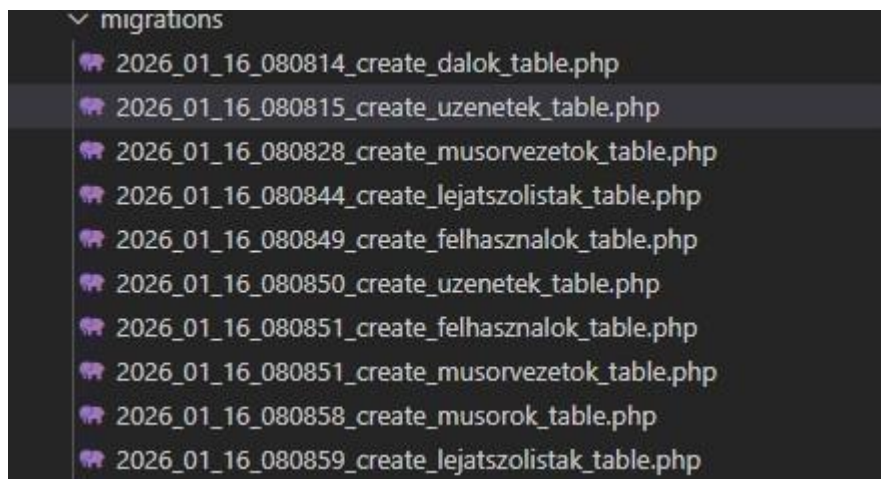
```
public function dal()  
{  
    return $this->belongsTo(Dal::class, 'dal_id');  
}
```

32. kép: BelongsTo függvény

- Néhol szükségünk volt belongsTo függvényt is használni.

A dal() egy modellfüggvény, amely egy kapcsolatot definiál az aktuális modell és a Dal modell között. A függvény meghívásakor megtudja, hogy az adott rekord egy dalhoz tartozik, amit a dal_id idegen kulcs azonosít. A függvény visszatérési értéke egy belongsTo kapcsolat. Ez nem egy sima üzleti logikát tartalmazó függvény, hanem egy kapcsolatdefiníció.

Migráció:



33. Kép: Migrációk

Az adatbázisunk minden adattáblájához csinálnunk kellett egy migrációt.

```
return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('uzenetek', function (Blueprint $table) {
            $table->id();
            $table->foreignId('felhasznalo_id')->references('id')->on('felhasznalok');
            $table->string('uzenet');
            $table->date('letrehozva');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('uzenetek');
    }
};
```

34. Kép: Migráció példa

Ez a migráció az uzenetek nevű adatbázistáblát hozza létre. A tábla célja, hogy eltárolja a felhasználók által létrehozott üzeneteket, és összekapcsolja őket a megfelelő felhasználóval.

Az `up()` függvény határozza meg, hogy milyen változtatások történjenek az adatbázisban a migráció futtatásakor.

Létrehoz egy új táblát `uzenetek` néven.

Egy automatikusan növekvő elsődleges kulcs (`id`) mezőt hoz létre.

Egy idegen kulcsot definiál, amely a `felhasznalok` tábla `id` mezőjére hivatkozik.

Ez biztosítja, hogy minden üzenet egy létező felhasználóhoz tartozzon.

Az üzenet szövegét tároló mező, szöveges (`VARCHAR`) formátumban.

Az üzenet létrehozásának dátumát tárolja.

Ebben az esetben nem a Laravel automatikus `created_at` mezőjét használja, hanem egy saját, kézzel kezelt dátummezőt.

Mit csinál a `down()` függvény?

Ez a függvény a migráció visszavonásáért felel.

Ha a migrációt rollbackeljük, az `uzenetek` tábla törlésre kerül az adatbázisból.

Miért hasznos ez backend szempontból?

Ez a migráció:

- egységesen létrehozza az üzenetek adatbázis-struktúráját,
- biztosítja az adatok közti kapcsolatot (felhasználó ↔ üzenet),
- lehetővé teszi az egyszerű telepítést és visszavonást,

- segít elkerülni a kézi adatbázis-módosításokból eredő hibákat.

Seeder:

Nekünk az adatbázis adatainak feltöltése minden tábla esetén migrációval történik

```
$dalok=
[
    [1, 'Queen', 'Bohemian Rhapsody', '00:05:55'],
    [2, 'Metallica', 'Nothing Else Matters', '00:06:28'],
    [3, 'Coldplay', 'Fix You', '00:04:55'],
    [4, 'Adele', 'Someone Like You', '00:04:45'],
    [5, 'Daft Punk', 'Get Lucky', '00:06:07'],
    [6, 'Pink Floyd', 'Wish You Were Here', '00:05:34'],
    [7, 'Nirvana', 'Come As You Are', '00:03:39'],
    [8, 'The Beatles', 'Let It Be', '00:04:03'],
    [9, 'U2', 'Beautiful Day', '00:04:08'],
    [10, 'Red Hot Chili Peppers', 'Under The Bridge', '00:04:24'],
    [11, 'Imagine Dragons', 'Radioactive', '00:03:06'],
    [12, 'AC/DC', 'Highway to Hell', '00:03:28'],
    [13, 'Eminem', 'Stan', '00:06:44'],
    [14, 'Madonna', 'Frozen', '00:06:12'],
    [15, 'Linkin Park', 'In The End', '00:03:36'],
    [16, 'Hans Zimmer', 'Time', '00:04:35'],
    [17, 'Lana Del Rey', 'Video Games', '00:04:42'],
    [18, 'Depeche Mode', 'Enjoy the Silence', '00:04:15'],
    [19, 'Radiohead', 'No Surprises', '00:03:49'],
    [20, 'Bon Jovi', 'Always', '00:05:53']
];
foreach($dalok as $key=>$value)
{
    Dal::create([
        'id'=>$value[0],
        'eloado'=>$value[1],
        'cim'=>$value[2],
        'hossza'=>$value[3]
    ]);
}
```

35. Kép: Seeder példa

Ez a kód előre definiált daladatokat tölt fel az adatbázisba a Dal modellen keresztül.

Tipikusan seedelésre (teszt- vagy mintaadatok feltöltésére) használjuk.

Mit csinálnak az egyes részek?

\$dalok = [...];

Egy tömb, ami a beszúrni kívánt dalokat tartalmazza (ID, előadó, cím, hossz).

foreach (\$dalok as \$key => \$value) Végigiterál a

dalokon egyesével. Dal::create([

Az create() metódusával új rekordot szúr be a dalok táblába.

'id' => \$value[0], 'eloado'

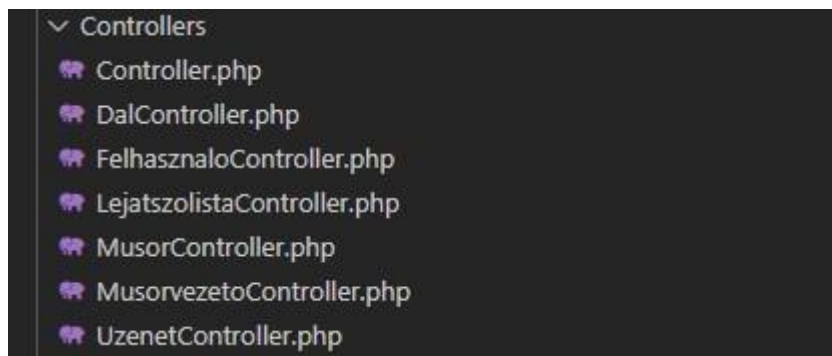
=> \$value[1],

'cim' => \$value[2],

'hossza' => \$value[3]

Az aktuális dal adatait rendeli hozzá az adatbázis mezőjéhez.

Controllerek:



36. Kép: Controllerek

Ez a kontroller a lejátszólisták backend oldali kezeléséért felel.

Az API-n keresztül biztosítja a CRUD műveleteket (lekérdezés, létrehozás, módosítás, törlés) a

Lejatszolista modellhez tartozó adatokon. Mit csinálnak egyes részei?? index():

- Lekéri az **összes lejátszólista rekordot**
- A kapcsolódó dalokat is betölti (with('dal'))

- Tömeges lekérdezéshez használható

```
public function index()
{
    return Lejatszalista::with('dal')->get();
}
```

37. Kép: Controller get függvény

getById(\$id):

- Egy lejátsszólistát kér le azonosító alapján
- Ha nem létezik, 404-es hibát ad vissza
- Egy konkrét elem megjelenítésére szolgál

```
public function getById($id)
{
    $lejatszalista=Lejatszalista::find($id);
    if(is_null($lejatszalista))
    {
        return response()->json(['Azonosító hiba:'=>'Nincs ilyen id-jű lejátsszólista.'],404);
    }
    return response()->json($lejatszalista,208);
}
```

38. Kép: Controller getById függvény

store(Request \$request):

- Új lejátsszólista létrehozása
- Ellenőrzi a kötelező mezők meglétét
- Sikeres mentés esetén új rekordot hoz létre az adatbázisban

```
public function store(Request $request)
{
    $validator=Validator::make($request->all(),
    [
        'dal_id'=>'required',
        'sorrend_szam'=>'required',
        'musor_id'=>'required'
    ]);
    if($validator->fails())
    {
        return response()->json(['Hiba:'=>'Fontos adat hiányzik.'],406);
    }
    $lejatszalista=Lejatszalista::create($request->all());
    return response()->json(['Új lejátsszólista létrehozva, a következő ID-vel:'=>$lejatszalista->id],201);
}
```

39. Kép: Controller store függvény

update(Request \$request, \$id):

- Meglévő lejátszólista módosítása
- Ellenőrzi, hogy létezik-e a megadott ID
- Validálja a beérkező adatokat
- Frissíti az adatbázisban lévő rekordot

```
public function update(Request $request, $id)
{
    $lejatszoLista=LejatszoLista::find($id);
    if(is_null($lejatszoLista))
    {
        return response()->json(['Hiba:'=>'Nem található a lejátszólista.'],404);
    }
    $validator=Validator::make($request->all(),
    [
        'dal_id'=>'required',
        'sorrend_szam'=>'required',
        'musor_id'=>'required'
    ]);
    if($validator->fails())
    {
        return response()->json(['Hiba:'=>'Fontos adat hiányzik.'],401);
    }
    $lejatszoLista->update($request->all());
    return response()->json(['Az alábbi lejátszólista adatai módosultak:'=>$lejatszoLista->id],201);
}
```

40. Kép: Controller update függvény

destroy(\$id):

- Lejátszólista törlése azonosító alapján
- Hibüzenetet ad, ha nem található
- Sikeres törlés esetén eltávolítja az adatot az adatbázisból

```
public function destroy($id)
{
    $lejatszoLista=LejatszoLista::find($id);
    if(is_null($lejatszoLista))
    {
        return response()->json(['Hiba:'=>'Nem található az adott ID-jű lejátszólista.'],404);
    }
    $lejatszoLista->delete();
    return response('Lejátszólista sikeresen törölve.',202);
}
```

41. Kép: Controller destroy függvény

getOlderShow(\$letrahozas):

- Egy megadott dátumnál **régebbi műsorokat** kér le
- Az adatbázisban a letrehozva mező alapján szűr (< \$letrehozás)
- Ellenőrzi, hogy létezik-e találat
- Találat esetén visszaadja a műsorok listáját
- Ha nincs találat, **404-es hibát** ad vissza megfelelő hibaüzenettel

```

}
public function getOlderShow($letrehozás)
{
    $musor=Musor::where('letrehozva','<',$letrehozás);
    if($musor->exists())
    {
        return $musor->get();
    }
    else
    {
        return response()->json(['Hiba:'=>'Nem található a megadott értéknél régebbi műsor.'],404);
    }
}

```

42. Kép: Controller getOlder függvény

getNewerShow(\$letrehozás):

- Egy megadott dátumnál **újabb műsorokat** kér le
- Az adatbázisban a letrehozva mező alapján szűr (> \$letrehozás)
- Ellenőrzi, hogy létezik-e találat
- Találat esetén visszaadja a műsorok listáját
- Ha nincs találat, **404-es hibát** ad vissza megfelelő hibaüzenettel

```

}
public function getNewerShow($letrehozás)
{
    $musor=Musor::where('letrehozva','>',$letrehozás);
    if($musor->exists())
    {
        return $musor->get();
    }
    else
    {
        return response()->json(['Hiba:'=>'Nem található a megadott értéknél újabb műsor.'],404);
    }
}
}

```

43. Kép: Controller getNewer függvény

Api végpontok:

Ezek Laravel keretrendszerben írt API-útvonalak (route-ok), amelyek a backendben meghatározzák, hogy mely HTTP-kérésekhez melyik vezérlő (controller) metódus tartozik.

Példa:

```

Route::get('/felhasznalok',[FelhasznaloController::class,'index']);
Route::get('/felhasznalok/{id}',[FelhasznaloController::class,'getById']->whereNumber('id'));
Route::post('/felhasznalok',[FelhasznaloController::class,'store']);
Route::put('/felhasznalok/{id}',[FelhasznaloController::class,'update']);
Route::get('/felhasznalok/filterby/{szerep}',[FelhasznaloController::class,'filterByRole']);
Route::get('/felhasznalok/searchbyname',[FelhasznaloController::class,'searchByName']);
Route::delete('/felhasznalok/{id}',[FelhasznaloController::class,'destroy']);

```

44. Kép: Api végpontok

Röviden: mind a felhasználók kezelésére szolgálnak — tipikus CRUD műveletek (Create, Read, Update, Delete).

Route::get('/felhasznalok', 'index')

→ Az összes felhasználót lekéri.

Route::get('/felhasznalok/{id}', 'getById')

→ Egy konkrét felhasználót ad vissza az id alapján.

Route::post('/felhasznalok', 'store')

→ Új felhasználót hoz létre .

`Route::put('/felhasznalok/{id}', 'update')`

→ Egy meglévő felhasználó adatait frissít.

`Route::get('/felhasznalok/filterby/{szerep}', 'filterByRole')`

→ Szerep (role) alapján szűri a felhasználókat.

`Route::get('/felhasznalok/searchbyname', 'searchByName')`

→ Név alapján keres a felhasználók között.

`Route::delete('/felhasznalok/{id}', 'destroy')`

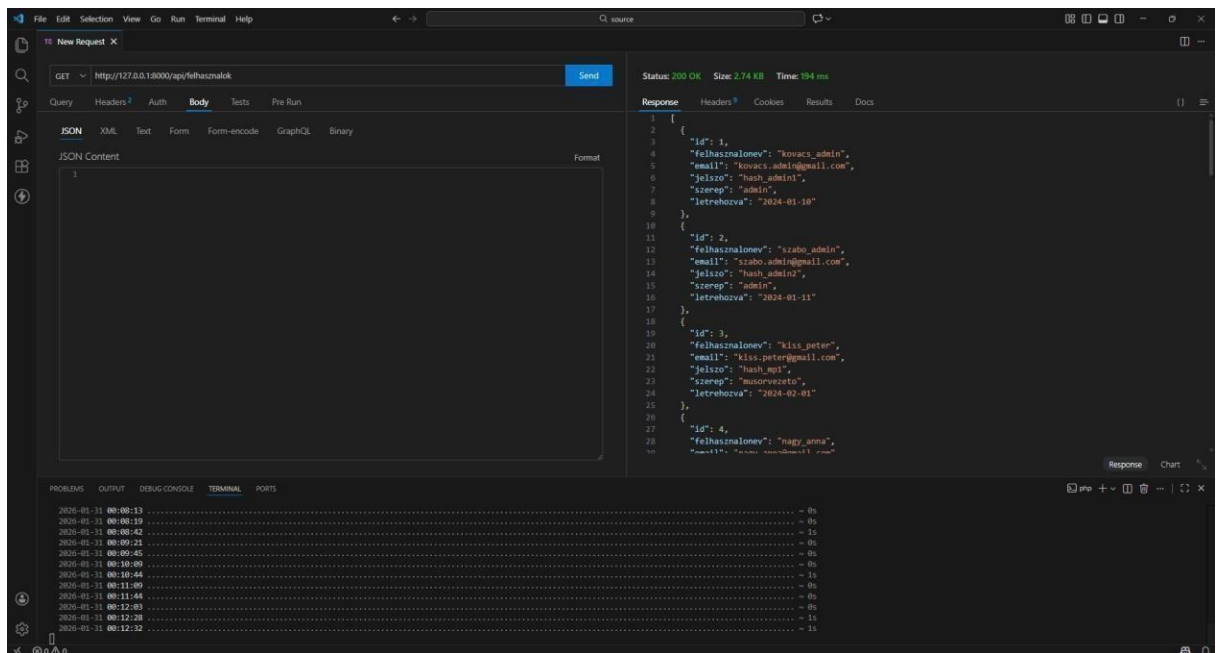
→ Törli a megadott felhasználót .

Tesztek:

Miután végeztünk a Backend egészének megírásával, elkezdődtek a tesztek. A tesztek közben azt vizsgáltuk, hogy az egyes lekérdezések a megfelelő értéket adják-e vissza, a megfelelő formátumban. Erre a Thunder Client nevezetű VS code-os bővítményt használtuk, amivel a backend api végpontokat tudtuk tesztelni.

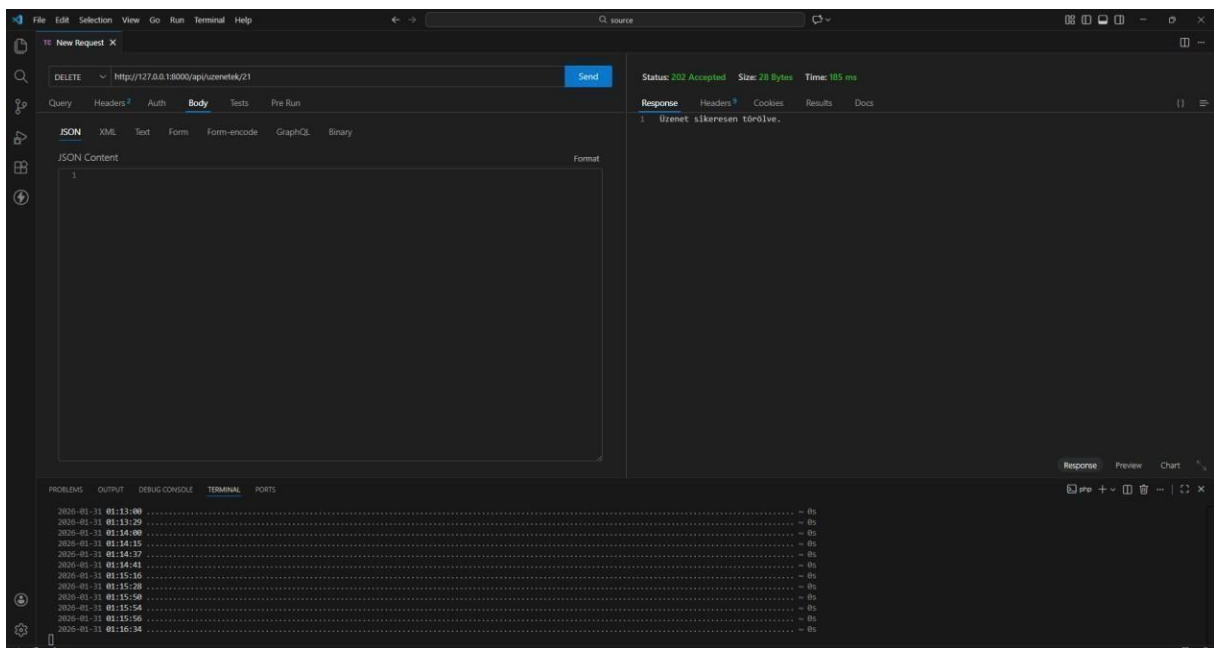
Erre pár példa:

Felhasználók get metódus sikeres teszt:



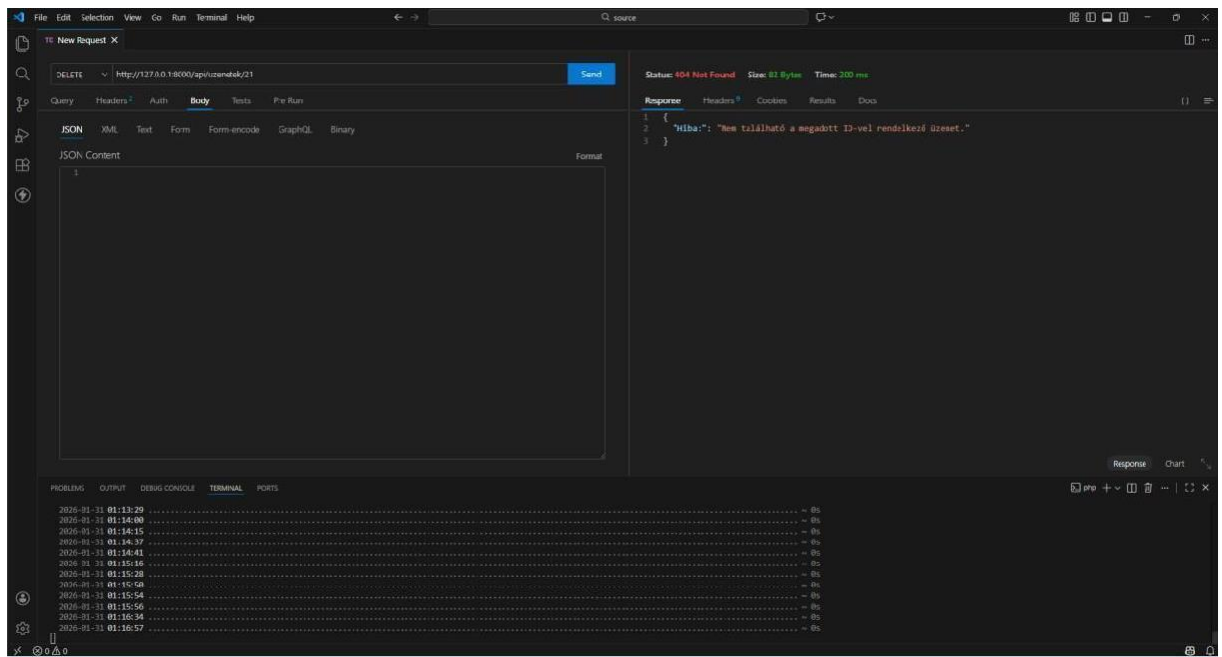
45. Kép: Get metódus sikeres teszt

üzenetek tábla Delete metódus sikeres teszt:



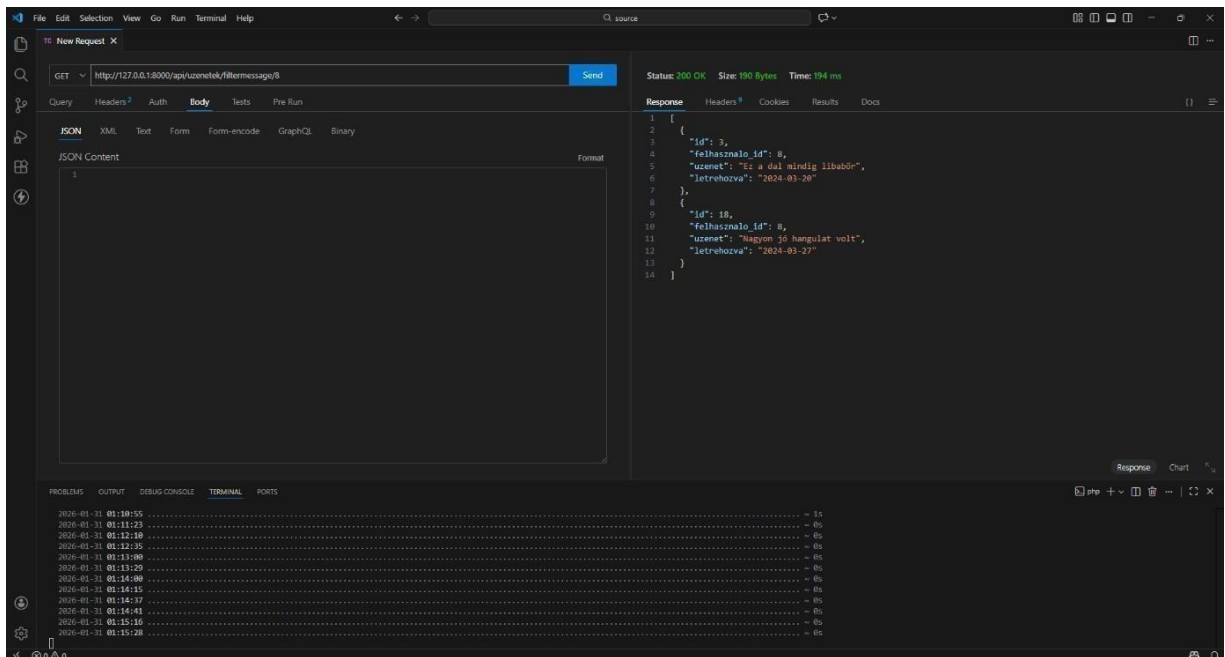
46. Kép: Delete metódus sikeres teszt

üzenetek tábla Delete metódus sikertelen teszt:



47. Kép: delete metódus sikertelen teszt

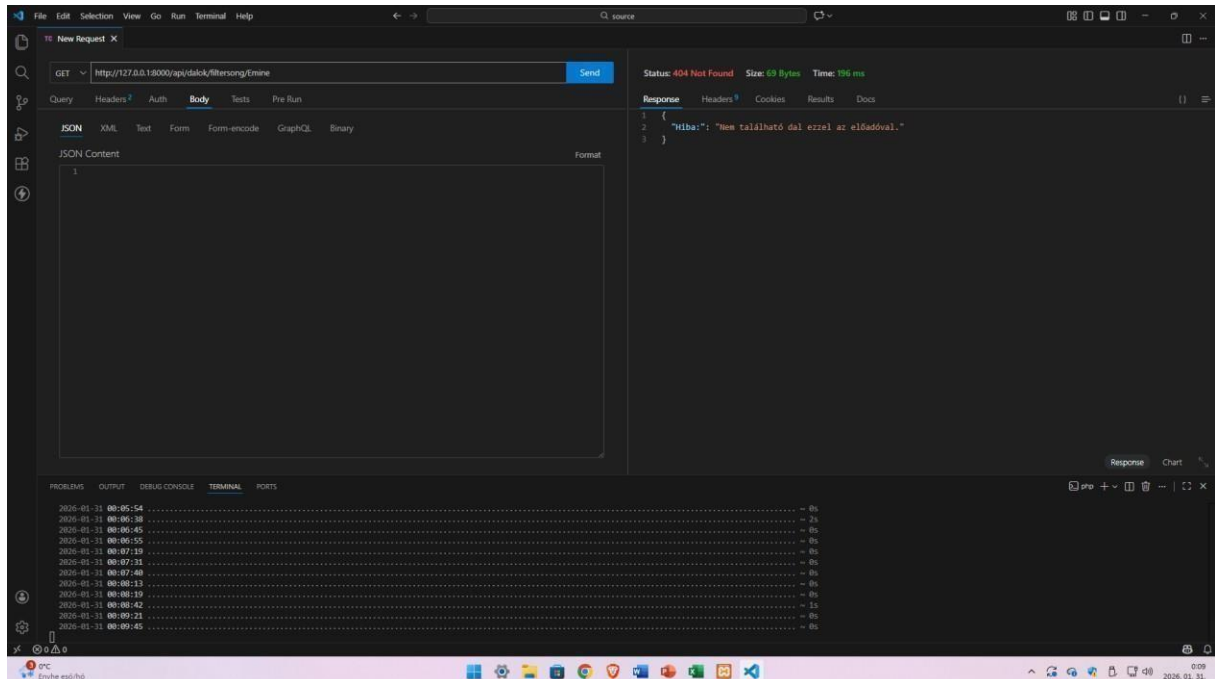
üzenetek tábla filterBy metódus sikeres teszt:



48. Kép: filterBy metódus sikeres teszt

üzenetek tábla

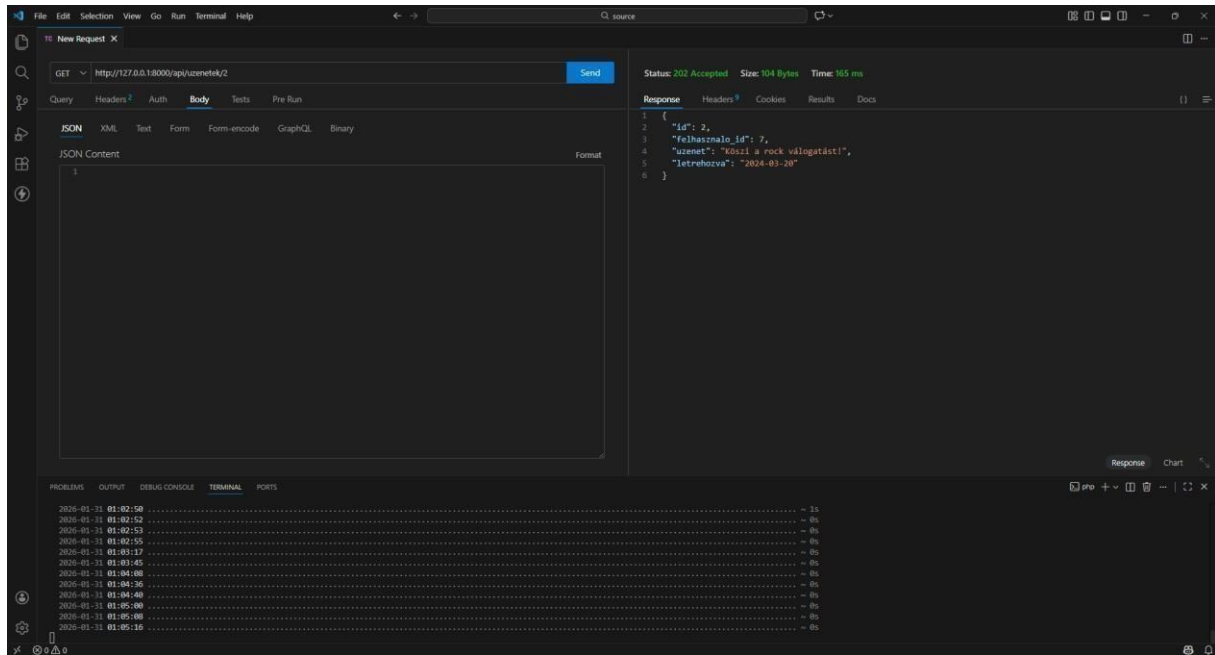
filterBy metódus sikertelen teszt:



49. Kép: filterBy metódus sikertelen teszt

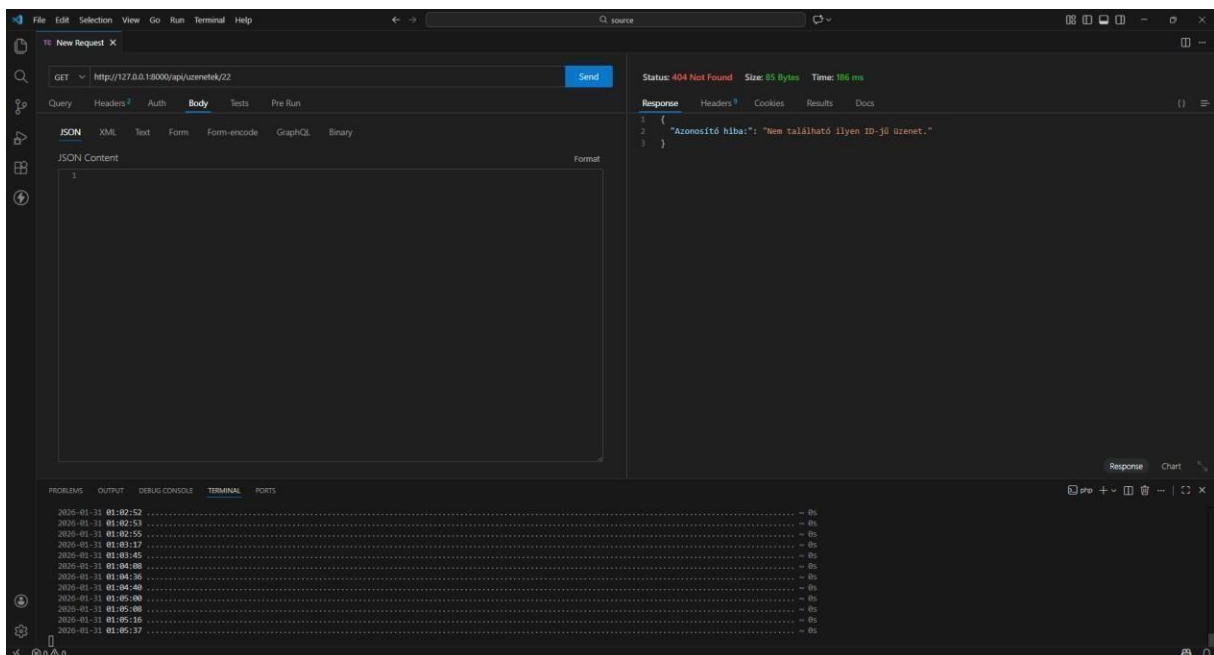
üzenetek tábla getById metódus sikeres teszt:

üzenetek tábla



50. Kép: *getById* metódus sikeres teszt

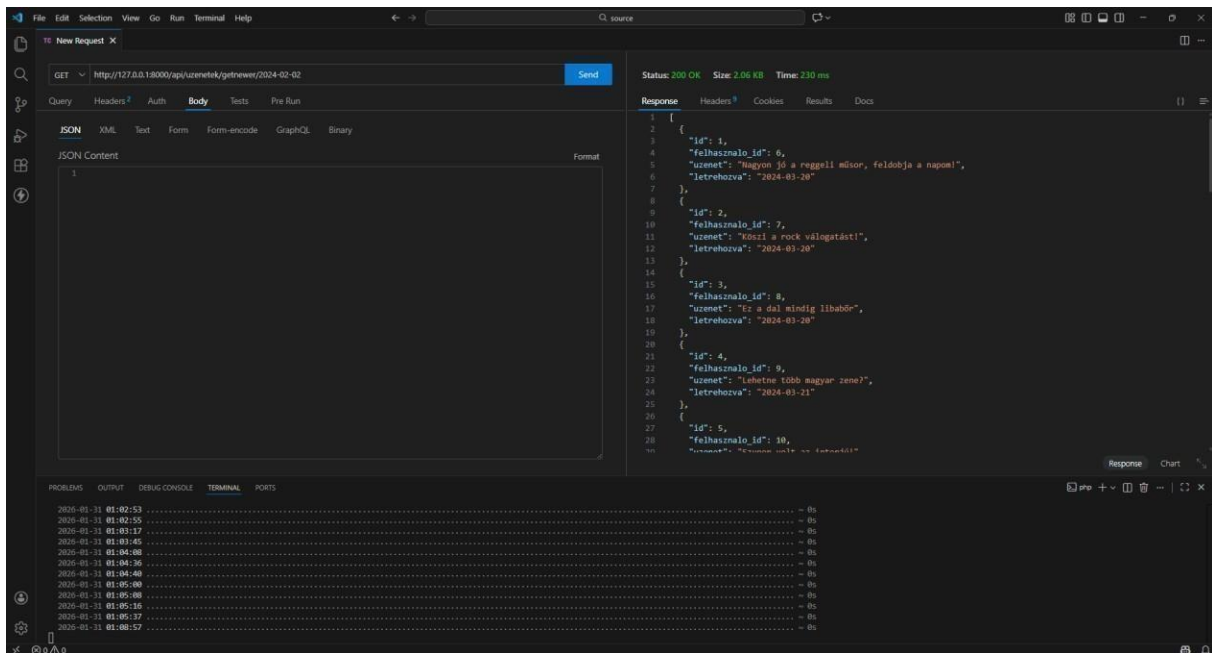
getById metódus sikertelen teszt:



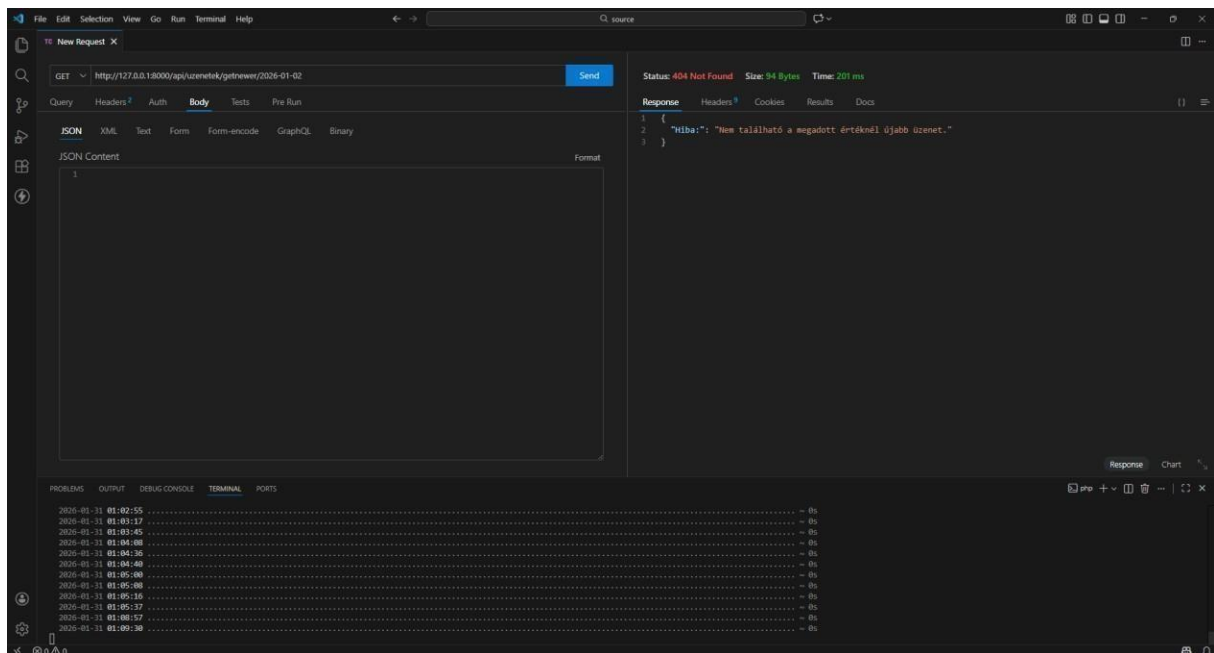
51. Kép: *getById* metódus sikertelen teszt

üzenetek tábla getnewer metódus sikeres teszt:

üzenetek tábla



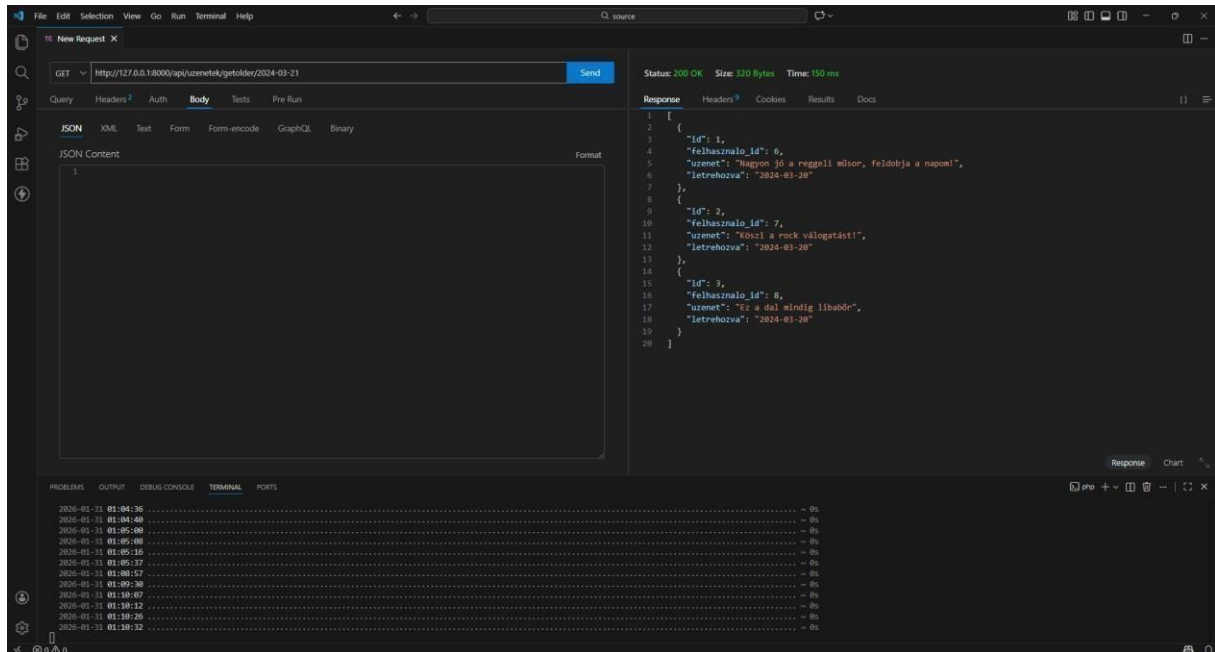
52. Kép: *getNewer* metódus sikeres teszt
getnewer metódus sikertelen teszt:



53. Kép: *getNewer* metódus sikertelen teszt

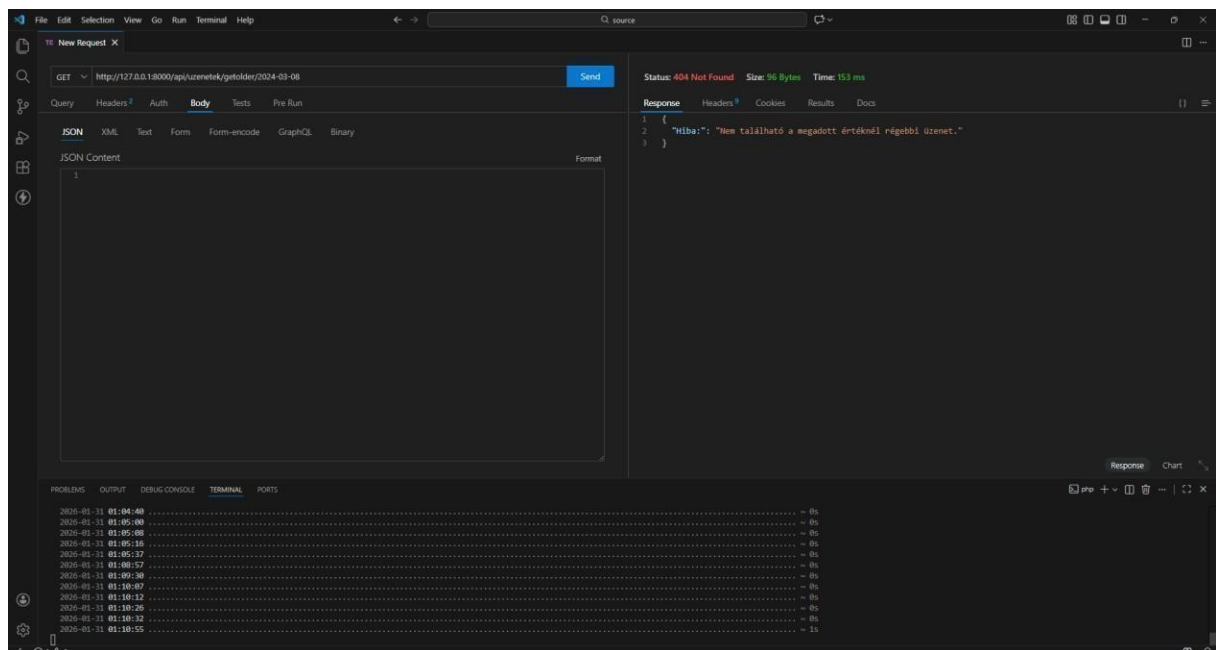
üzenetek tábla

üzenetek tábla getolder metódus sikeres teszt:



54. Kép: getOlder metódus sikeres teszt

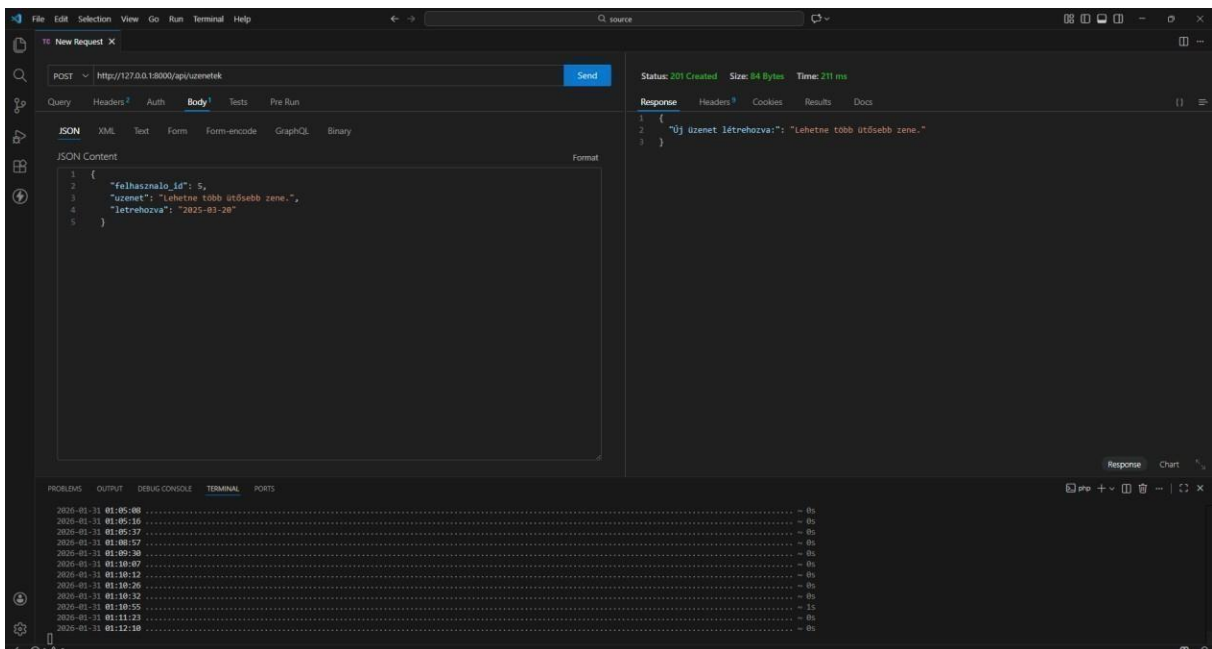
getolder metódus sikertelen teszt:



55. Kép: getOlder metódus sikertelen teszt

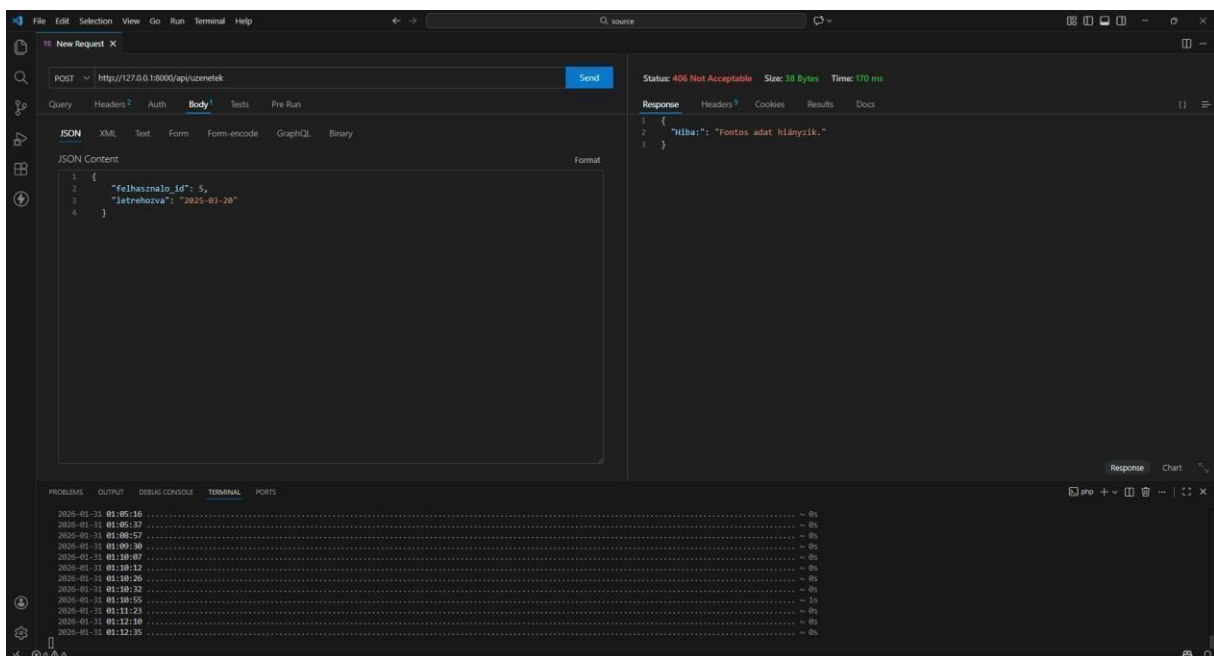
üzenetek tábla

üzenetek tábla post metódus sikeres teszt:



56. Kép: post metódus sikeres teszt

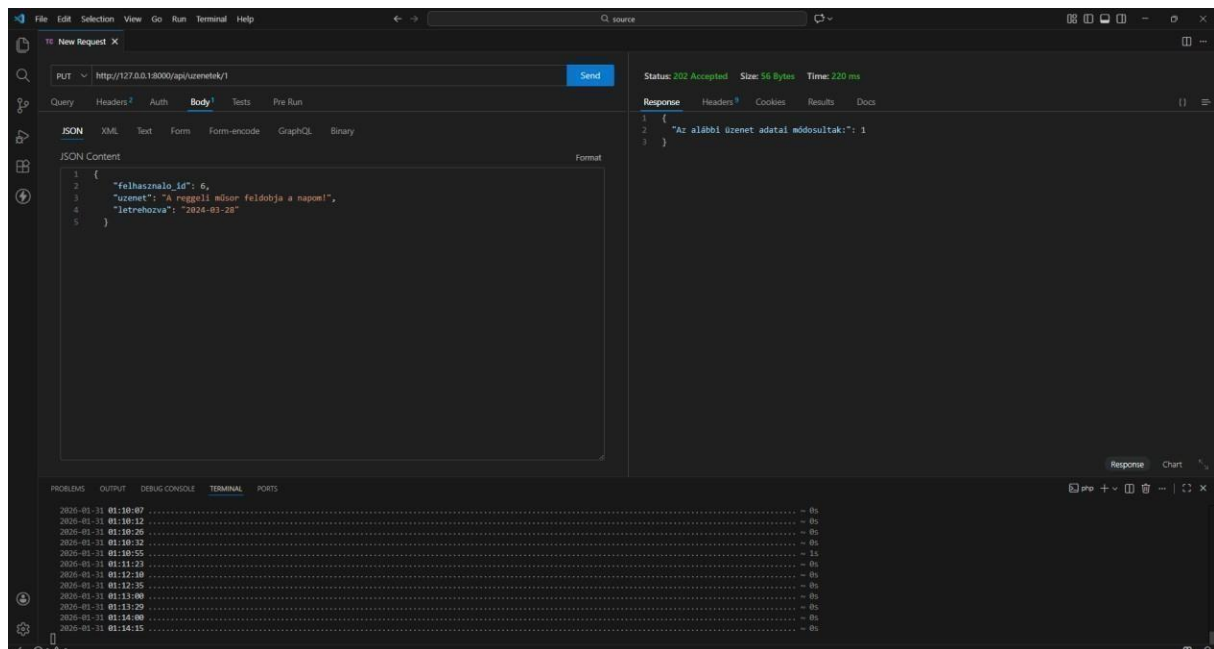
post metódus sikertelen teszt:



57. Kép: post metódus sikertelen teszt

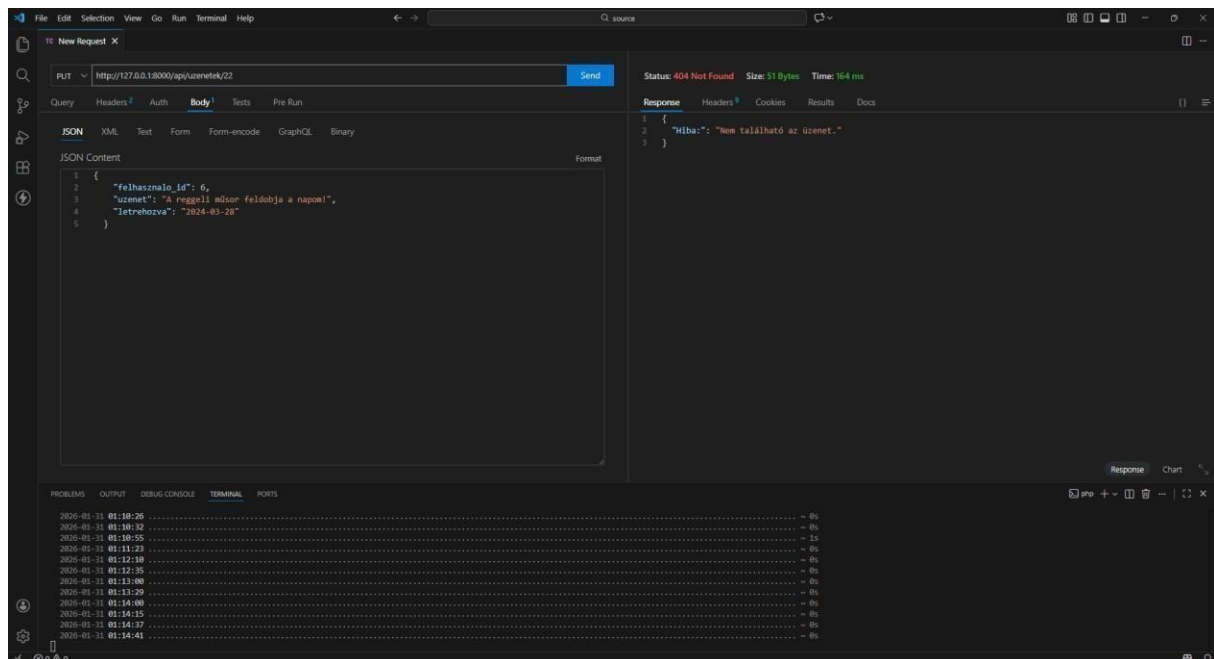
üzenetek tábla

üzenetek tábla put metódus sikeres teszt:



58. K\u00e9p: put met\u00f3dus sikeres teszt

put met\u00f3dus sikertelen teszt:



59. K\u00e9p: put met\u00f3dus sikertelen teszt

Tesztelés

A fejlesztés során kiemelt figyelmet kapott az alkalmazás megbízhatóságának biztosítása. Ennek érdekében a frontend működését kétféle módon ellenőriztük: automatizált tesztekkel és manuális teszteléssel.

Automatizált tesztelés

Az alkalmazás egyes részeinek ellenőrzésére egységteszteket alkalmaztunk. A tesztelés célja az volt, hogy biztosítsuk az egyes komponensek és szolgáltatások helyes működését, valamint a hibák korai felismerését. Az egységtesztek megvalósításához a Vitest tesztkeretrendszert használtuk. Ez egy modern és gyors tesztelő eszköz, amely lehetővé teszi a hatékony tesztfuttatást és az egyszerű konfigurációt. A tesztek futtatása az `ng test` paranccsal történik. A futtatás során az eredmények közvetlenül a terminálban jelennek meg, így nincs szükség külön grafikus felületre. A tesztelő környezet figyelési módban működik, tehát a fájlok módosítása esetén automatikusan újra futtatja a teszteket. Az alkalmazás főbb egységei külön-külön tesztelésre kerültek:

Komponensek:

- Footer komponens
- Navbar komponens
- Home oldal
- Player komponens
- Comments komponens
- Sidebar (bal és jobb oldali)

Szolgáltatások:

- PlaylistService
- MusicService
- CommentService

A tesztek ellenőrzik például:

- a komponensek létrejöttét
- a szolgáltatások működését
- az alapvető logikai folyamatokat

A tesztelés eredményeként 12 tesztből 12 sikeresen lefutott. A tesztek gyorsan és hibamentesen működtek, ami azt mutatja, hogy az alkalmazás főbb részei stabilan lettek implementálva. Tesztfuttatás eredménye:

```
DEV v4.0.18 C:/Users/User/Desktop/FRONTEND/FRONTEND
✓ SzikszifM src/app/footer/footer.spec.ts (1 test) 166ms
✓ SzikszifM src/app/playlist-service.spec.ts (1 test) 7ms
✓ SzikszifM src/app/player/player.spec.ts (1 test) 196ms
✓ SzikszifM src/app/music-service.spec.ts (1 test) 4ms
✓ SzikszifM src/app/comment-service.spec.ts (1 test) 6ms
✓ SzikszifM src/app/comments/comments.spec.ts (1 test) 185ms
✓ SzikszifM src/app/sidebar-left/sidebar-left.spec.ts (1 test) 232ms
✓ SzikszifM src/app/sidebar-right/sidebar-right.spec.ts (1 test) 242ms
✓ SzikszifM src/app/navbar/navbar.spec.ts (1 test) 205ms
✓ SzikszifM src/app/home/home.spec.ts (1 test) 324ms
✓ should create 322ms
✓ SzikszifM src/app/app.spec.ts (2 tests) 266ms

Test Files 11 passed (11)
Tests 12 passed (12)
Start at 18:50:44
Duration 3.16s (transform 882ms, setup 7.21s, import 2.02s, tests 1.83s, environment 14.55s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

60. Kép: Tesztfuttatás eredménye

Manuális tesztelés

A frontend működését manuálisan is ellenőriztük, különös figyelmet fordítva a felhasználói interakciókra és az űrlapok működésére. A manuális tesztelés során azt vizsgáltuk, hogy az alkalmazás megfelelően reagál-e különböző bemenetekre, különösen hibás vagy hiányos adatok esetén.

Bejelentkezési űrlap tesztelése

A bejelentkezési oldalon több különböző esetet vizsgáltunk.

Ha a felhasználó üresen hagyja az email vagy jelszó mezőt, az alkalmazás hibaüzenetet jelenít meg. Ez biztosítja, hogy a kötelező mezők ne maradjanak kitöltetlenül.

The screenshot shows a web application interface with a dark purple header and footer. The header contains navigation links: Kezdőlap, Rólunk, Műsorvezetők, Műsorok, and a Bejelentkezés button. The main content area features a white registration form titled 'Bejelentkezés' with the subtitle 'Lépj be és kezd el a saját lejátszási listádat.' The form has two input fields: 'Email cím' and 'Jelszó'. The 'Email cím' field is highlighted with a red border, and a red error message 'Az email cím megadása kötelező.' is displayed below it. The 'Jelszó' field is empty. Below the input fields is a blue 'Bejelentkezés' button. At the bottom of the form, there are links: 'Nincs még fiókod? Regisztráció' and 'Vissza a főoldalra'. The footer contains the copyright notice '© 2025 Széchenyi István Katolikus Technikum és Gimnázium | Minden jog fenntartva' and the text 'Székely FM'.

61. Kép: Üres mezők hibakezelése

Ha a felhasználó hibás email formátumot ad meg, a rendszer figyelmezteti, hogy érvényes email címet kell megadni.

The screenshot shows the same web application interface as the previous one. The 'Email cím' field now contains the text 'asd' and is highlighted with a red border. A red error message 'Adj meg egy valid email címet.' is displayed below the field. The 'Jelszó' field remains empty. The 'Bejelentkezés' button and the links at the bottom of the form are still visible. The footer contains the same copyright notice and 'Székely FM' text.

62. Kép: Hibás email formátum

Ha a jelszó nem megfelelő hosszúságú, a rendszer szintén hibaüzenetet jelenít meg.

63. Kép: Rövid jelszó hiba

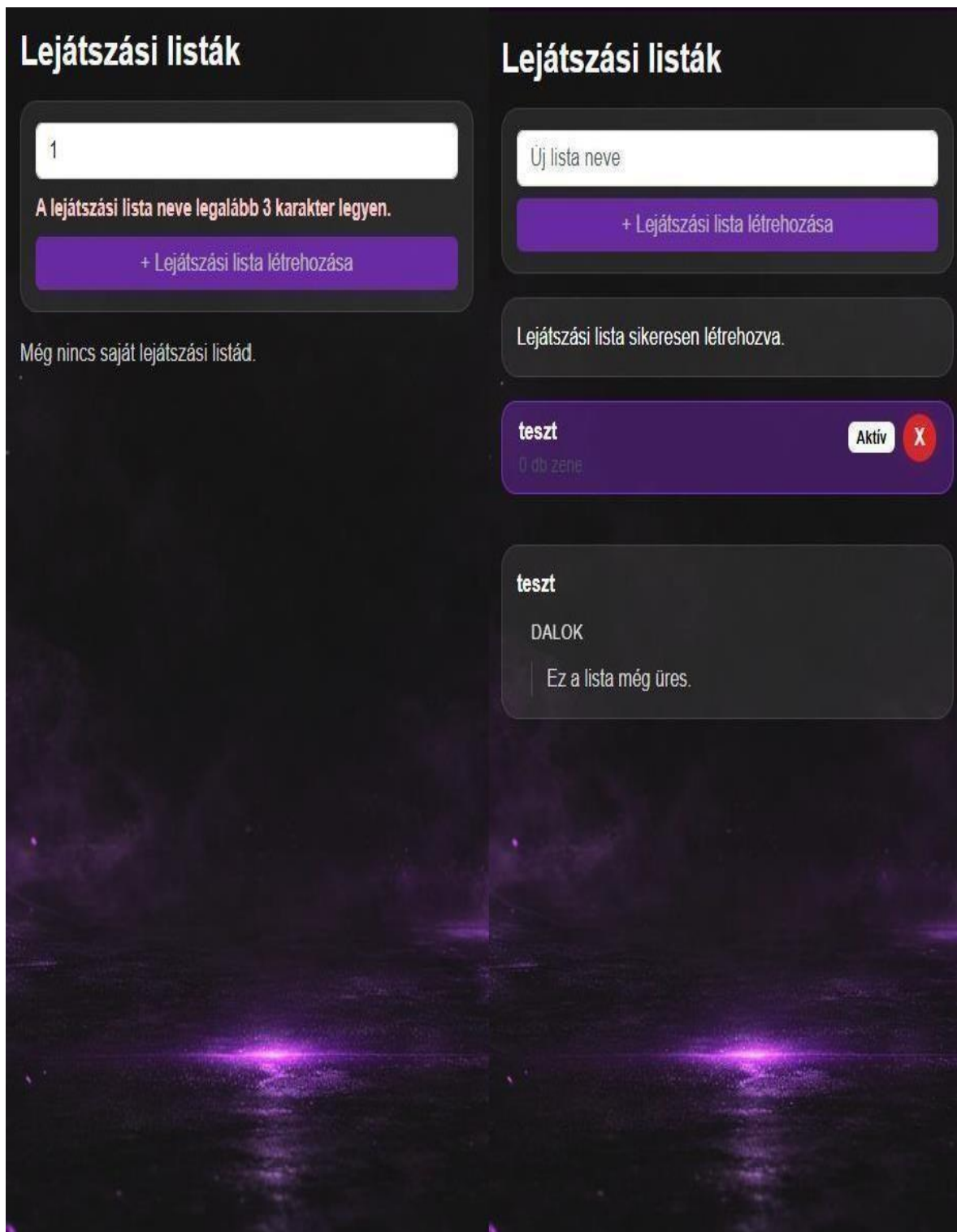
Sikeres adatok megadása esetén az űrlap elküldhető, és a rendszer feldolgozza a bejelentkezést.

64. Kép: Sikeres bejelentkezés

Playlist funkciók tesztelése

- a playlist kezelés során ellenőriztük a következőket: □
- új playlist létrehozása működik
- üres név esetén nem engedi □ létrehozni

- törlés után azonnal frissül a ☐ lista
- dal hozzáadás után ☐ megjelenik a playlistben ☐

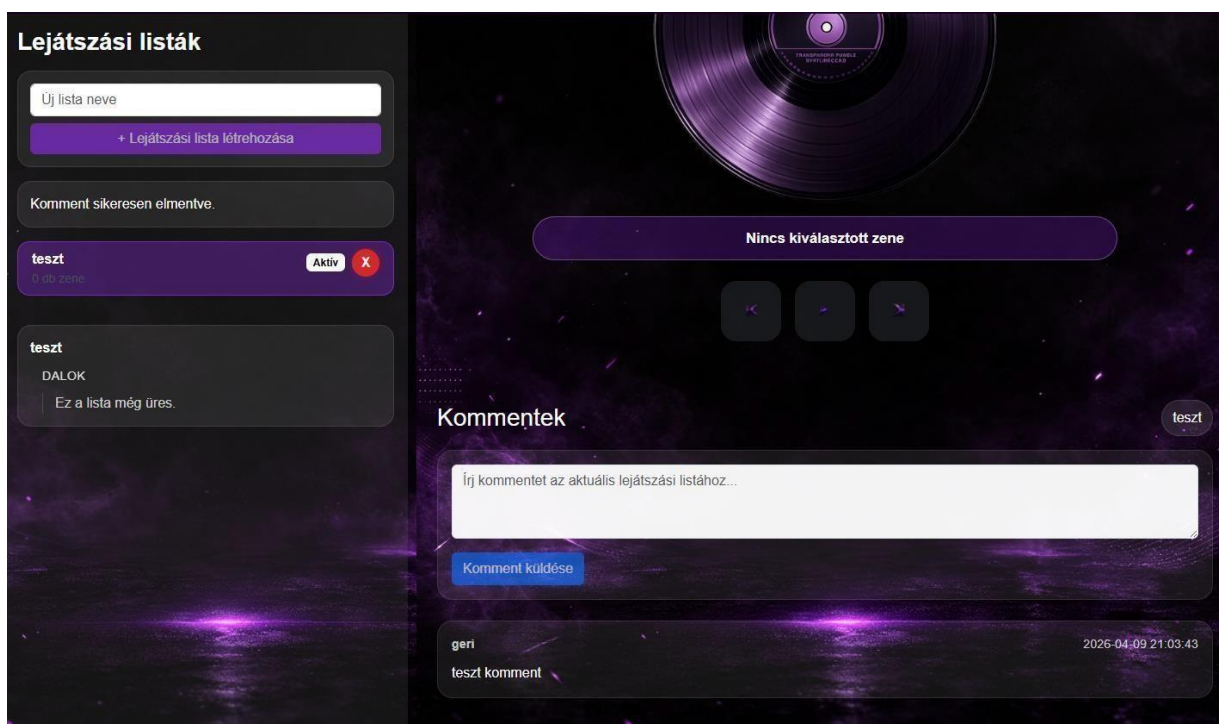


65. Kép: Rövid playlist név 66. Kép: Playlist létrehozás

Kommentelés tesztelése

A komment funkció esetében a következőket vizsgáltuk:

- nem lehet üres kommentet küldeni□
- csak bejelentkezett felhasználó tud kommentelni□
- sikeres komment után azonnal megjelenik□



67. Kép: Komment hozzáadás

Továbbfejlesztési lehetőségek

A projekt jelenlegi állapotában már képes ellátni az alapvető funkciókat, azonban több olyan fejlesztési lehetőség is van, amelyekkel az alkalmazás tovább bővíthető, modernebbé és felhasználóbarátabbá tehető. Ezek a továbbfejlesztések nemcsak a felhasználói élményt javíthatják, hanem technikai szempontból is stabilabbá és skálázhatóbbá tehetik a rendszert. Az egyik legfontosabb továbbfejlesztési lehetőség a felhasználói felület vizuális továbbfejlesztése. Bár az alkalmazás jelenlegi formájában is jól használható, később érdemes lenne modernebb, még letisztultabb megjelenést kialakítani. Ide tartozhatna az egységesebb színvilág, részletesebb hover effektek, animációk, valamint a mobilos nézet további finomítása. Ezek a változtatások nem feltétlenül módosítják a működést, de jelentősen javíthatják az összehatást. További fejlesztési lehetőség a valódi zenelejátszás beépítése. A jelenlegi player inkább a kiválasztott dal kezelésére és a vezérlőfelület megjelenítésére szolgál, viszont később érdemes

lenne tényleges audíolejátszást is megvalósítani. Így a felhasználó nemcsak kiválaszthatná a zenéket, hanem azokat közvetlenül meg is hallgathatná az alkalmazáson belül. Ezzel a rendszer sokkal közelebb kerülne egy valódi online rádiós vagy zenei platform működéséhez. A playlist kezelés is tovább bővíthető lenne. Jelenleg a felhasználó létrehozhat, kiválaszthat és törölhet playlisteket, illetve dalokat adhat hozzájuk, de később hasznos lehetne a playlist-ek átnevezésének lehetősége, a dalok sorrendjének kézi módosítása, valamint a kedvencek jelölés funkció. Ezek a bővítések nagyobb szabadságot adnának a felhasználóknak saját listáik kezelésében. A kommentrendszer szintén bővíthető lenne. Jelenleg a felhasználók kommentet írhatnak az adott playlisthez, de később lehetőség nyílhatna a kommentek szerkesztésére és törlésére, valamint akár válaszok írására is. Ezzel egy összetettebb, közösségibb funkció alakulhatna ki, amely növeli a felhasználói aktivitást. Hasznos bővítés lehetne továbbá a felhasználói profil oldal bevezetése is. Ezen a felhasználó megtekinthetné saját adatait, módosíthatná a profilinformációkat, esetleg nyomon követhetné saját playlistjeit vagy aktivitását. Ez személyesebbé tenné az alkalmazást, és jobban elkülönítené az egyes felhasználókhoz tartozó tartalmakat. Összességében elmondható, hogy a projekt jó alapot biztosít további fejlesztésekhez. A jelenlegi rendszer már működőképes és jól strukturált, ezért a későbbi bővítések könnyebben megvalósíthatók. A legfontosabb jövőbeli irányok a felhasználói élmény javítása, a funkciók bővítése, valamint a rendszer technikai továbbfejlesztése lehetnek.

Irodalomjegyzék

A projektünk elkészítéséhez az alábbi forrásokat használtuk:

- • <https://angular.dev/>
- • <https://getbootstrap.com/>
- https://en.wikipedia.org/wiki/Front-end_web_development □
- <https://laravel.com/> □